Check for updates

Concurrency and Computation: Practice and Experience

WILEY

RESEARCH ARTICLE

Bridging Functional Encryption and Secure Function Evaluation: Toward Efficient and Coercion-Resistant Multi-Party Systems

Rahat Naz[1] | Jaydeep Howlader[1] | Shahnawaz Ahmad[2]

[1]Department of Computer Science and Engineering, National Institute of Technology Durgapur, Durgapur, West Bengal, India | [2]School of Computer Science Engineering and Technology, Bennett University, Greater Noida, India

Correspondence: Shahnawaz Ahmad (shahnawaz98976@gmail.com)

Received: 11 December 2025 | **Revised:** 4 August 2026 | **Accepted:** 28 August 2026

Keywords: coercion resistance | functional encryption | privacy-preserving computation | secure function evaluation | secure multi-party computation

ABSTRACT

In this study, a hybrid FE–SFE framework that combines distributed key generation, input masking, authenticated secret sharing, active verification, GPU acceleration, batching, compression and asynchronous communication was developed. The implementation was tested on a 10-node CPU-GPU cluster with Adult, MNIST and controlled benchmark vectors, 1000–10,000 records, and various security levels, with 2–10 parties. The framework maintained exact functional correctness across 287,000 valid evaluations, and all 80,000 injected malicious deviations were incorrectly evaluated without producing invalid output. The end-to-end latency was reduced by 41.06%–41.81% compared to sequential execution, and four-stream GPU encryption achieved 18,234 records/s. The overhead of the active verification and coercion protection was moderate. The communication volume decreased by 62.70% when the security metadata was added. The strong scaling efficiency remained above 81% up to 8 nodes, but dropped below the desired 75% at 10 nodes. The proposed framework provides a viable basis for secure multi-party analytics under controlled, security-aligned experimental conditions that can be used for rigorous and reproducible evaluations, while also being functional, verified, coercion-resistant, distributed, and performant.

1 | Introduction

Distributed analytics is increasingly in demand across multiple organizations, devices, or computational nodes to process sensitive information without revealing the underlying data. It is clear from the above that the participating entities may have the same analytical goal but are not trusted by each other, such as vertical federated learning, blockchain-based collaboration, cyber-risk prediction, and smart-grid monitoring [1–4]. Conventional access control ensures the security of stored data, but it is not very secure when plaintext is released for computation. Privacy-preserving computation, on the other hand, aims to control the information that can be inferred from encrypted or distributed inputs during analysis. In the realm of federated learning, functional-encryption-based FL has shown that it is possible to evaluate selected functions without revealing the entire client data, but it remains to be seen how well it can be put into practice in practice, as efficient key management, ciphertext processing, and multi-party coordination are still required [5].

In addition to traditional encryption, there is also fine-grained functional encryption (FE). An authorized functional key will not reveal the output of an unauthorized function, but will reveal only the output of the authorized function. There have been recent advances that have enhanced the flexibility of this model, such as updatable ciphertexts, ciphertext-policy

© 2026 John Wiley & Sons Ltd.
All rights reserved, including rights for text and data mining and training of artificial intelligence technologies or similar technologies.

authorization, decentralized key management, and function-hiding inner-product constructions [6–8]. The advances are especially significant for distributed learning and encrypted analytics, where they reduce unnecessary data disclosure and preserve application-specific utility. However, FE is more appropriate for limited-function families than for general collaborative computation. Homomorphic processing offers more expressive encrypted processing, but still has significant implementation costs due to bootstrapping, ciphertext expansion, and large-integer arithmetic. Some of these penalties have been mitigated by packed-message TFHE and CKKS-based integer computation, but there are still memory movement and repeated encrypted operations that are important performance constraints [9, 10].

Secure function evaluation and secure multi-party computation (SMPC) are related, but distinct problems. They enable multiple parties to assess the role of private inputs without revealing excessive information during protocol execution. The interaction and circuit-processing overhead have been minimized by optimized Tiny-OT and half-gate constructions, and the analysis has been extended to legally enforceable fairness, going beyond confidentiality [11, 12]. Another advantage of proactive secret-sharing systems is that the distributed shares can be updated at lower setup and communication costs [13]. But the pre-processing, synchronization, authenticated reconstruction, and malicious-party verification issues still hinder the practical deployments of SMPC. Verifiable outsourcing and encrypted execution frameworks address output integrity, but verification itself must be carefully integrated to avoid eliminating the benefits of accelerated computation [14–16].

A broader group of privacy-preserving systems has improved specific operations, including private set intersection, encrypted range search, searchable data sharing, and federated learning. Feature-retrieved private set intersection reduces disclosure during cross-party matching, while forward-private set intersection limits the exposure created by later updates [17, 18]. Fully private multidimensional range queries and multi-owner searchable encryption similarly support controlled access to encrypted repositories [19, 20]. In federated environments, adaptive CKKS segmentation and packed homomorphic encryption have improved computational efficiency by amortizing ciphertext operations across several values [21, 22]. These methods are valuable within their intended problem domains, but they do not by themselves provide a unified mechanism for function authorization, arbitrary secure evaluation, malicious-behavior detection, and post-execution coercion resistance.

The remaining challenge is therefore architectural rather than purely cryptographic. The distributed key generation, function-restricted disclosure, authenticated secret sharing, secure evaluation, verification, reconstruction, and state protection must be integrated into a practical system without adding excessive computation or communication overhead. While federated distributed key generation does not require a central authority, efficient coordination among the nodes participating in the system is important [23]. At the implementation level, side-channel-sensitive circuit operations and hardware-dependent memory behavior can also undermine apparently efficient designs [24]. Function hiding reduces information leakage from the evaluated operation, while behavior-aware cryptographic intelligence emphasizes the importance of observing protocol misuse and anomalous execution rather than treating confidentiality as the only security objective [25–27].

This study addresses these limitations through a hybrid FE–SFE framework designed for distributed CPU–GPU environments. The distributed key generation, function-restricted disclosure, authenticated secret sharing, secure evaluation, verification, reconstruction, and state protection must be integrated into a practical system without adding excessive computation or communication overhead. While federated distributed key generation does not require a central authority, efficient coordination among the nodes participating in the system is important [23].

The principal contributions are as follows:

1. A unified FE–SFE protocol that combines function-restricted disclosure, authenticated secure computation, malicious security with abort, and bounded post-execution coercion resistance.
2. A multithreaded and GPU-accelerated implementation that achieves an encryption-stage throughput of 18,234 records/s and reduces complete-protocol latency by 41.06%–41.81% relative to sequential execution.
3. A communication-optimized execution pipeline using batching, compression, and asynchronous transfer, reducing 10-party communication by 62.70% after verification and coercion-related metadata are included.
4. A controlled evaluation of functional correctness, malicious deviations, coercion-state distinguishability, component ablation, resource consumption, statistical significance, and security-aligned baselines.
5. An experimentally bounded scalability analysis showing efficient strong scaling through eight nodes, followed by a saturation region at 10 nodes when parallel efficiency falls below the predefined 75% threshold.

The rest of the paper is organized as follows: Section 2 covers the topics of FE, SMPC, function hiding, verifiable computation, and privacy-preserving distributed systems. The architecture, protocol construction, assumptions about the adversary, implementation approach, and evaluation process are described in Section 3. The results of the experiments and statistical analysis are presented in Section 4. The implications and limitations are discussed in Section 5, and the study is concluded, and future directions of research are outlined in Section 6.

2 | Literature Review

2.1 | Functional Encryption and Homomorphic Computation

Functional encryption has become a viable way to restrict access to data to the result of a legitimate computation. In recent years, constructions have shifted from a central management to

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

decentralized and delegatable constructions. Qian et al. [28] introduced a decentralized multi-client functional encryption for inner products, showcasing its application in federated learning with multiple clients providing encrypted features. Han et al. [29] also studied the decentralized inner-product functional encryption, focusing on eliminating the need for a central key authority. Han et al. [30] addressed delegation by introducing a DFE-IP, which enabled functional capabilities to be transferred without compromising the confidentiality of the underlying vectors. These studies combined enabled greater flexibility in authorizations but were still limited to inner-product operations.

The homomorphic encryption has a wider range of encrypted arithmetic but is accompanied by the expansion of the ciphertext and the need to process the polynomial. Che et al. [31] reduced the multiplication overhead in multi-key homomorphic encryption by shrinking RGSW ciphertexts without using traditional relinearization during multiplication. Cai et al. [32] demonstrated how to use multi-key homomorphic encryption to secure FL, in which locally encrypted data on each client can be aggregated without revealing any local data. Xia and Wang [33] implemented compact full-domain functional bootstrapping for one of the costliest phases of lattice-based encrypted computation at the hardware level.

2.2 | Secure Multi-Party and Private Function Evaluation

Secure Multi-party Computation enables multiple parties to jointly process data without revealing their own data. To reduce some of the representation costs of the traditional circuit-based approach, Domingo-Ferrer and Manjón [34] introduced circuit-free general-purpose computation by outsourcing the computation of a function in a co-utile, unlinkable manner. van Daalen et al. [35] designed an n-party protocol for distributed privacy-preserving analytics, which was developed for scalar-product workloads. These methods work well for a specific operation, but can be limited in their applicability when the application involves a combination of arithmetic, conditional logic, or hidden functions.

Private function evaluation is a technique for hiding both the function being evaluated and its input. Ji et al. [36] considered multi-party private function evaluation for RAM programs, in which data-dependent access patterns and function confidentiality were modeled using a more expressive computational model. Liu et al. [15, 16] studied the privacy-preserving outsourced computation of multiple users on Boolean circuits, focusing on the input privacy and delegated execution. Yang et al. [37] proposed Fregata, a single protocol that combines secure 2-party protocols for fast private inference and the cost of switching between cryptographic subprotocols. There have been other attempts to quantum alternatives, such as Cao [38], who proposed quantum private function evaluation, and Mohanty and Debnath [39], who introduced an information-theoretically secure quantum protocol for multi-party private set intersection. While important, quantum protocols remain of theoretical interest, and the communication infrastructure and devices required for them are not available in most conventional distributed deployments.

2.3 | Verifiability, Robustness, and Secret Management

Confidentiality is not enough if there is a possibility that a participant or outsourced evaluator could go off script. In this work, Chen and Zhang [40] introduced publicly verifiable homomorphic secret sharing for polynomial evaluation, which allows anyone to verify the validity of the computation without being able to reconstruct the private inputs. Li et al. [41] adopted a bulletin-board structure for secure multi-party computation that is publicly verifiable, and Gai et al. [42] enhanced robustness by making the multi-party results publicly verifiable. To further explore the verification of multi-source outsourced data, Zhang et al. [43] proposed a polynomial-based verification that allows for verification without exposing source records.

It is also important to have flexibility in the management of secrets for long-running systems. Harn et al. [44] introduced a secret sharing scheme with an adjustable threshold, allowing the reconstruction threshold to be adjusted without rebuilding the entire secret sharing environment. These mechanisms can be helpful if membership and/or trust expectations change. However, most of the time, the study of threshold adjustment, public verification and secure computation is considered separately. The synergy among the three is not as well studied as that of the two mentioned above.

2.4 | Application-Oriented Privacy-Preserving Systems

There have been a number of recent studies that have modified the privacy-preserving computation to the application domain. Yang et al. [45] introduced an oblivious pseudorandom function-based scalable delegated multi-party private set intersection and cardinality computation. Wang et al. [18, 19] studied private federated trajectory matching, in which location sequences need to be matched without revealing full mobility histories. When multiple users update or query encrypted indexes, the integrity of the indexes is crucial, especially when some users are corrupted. Wang et al. [12, 46, 47] discussed verifiable dynamic multi-user searchable encryption in the presence of corrupted users. To address the issue of unauthorized access to the metaverse, Zhang et al. [48] proposed a new approach to key-abuse resistance in decentralized ciphertext-policy attribute-based encryption for data sharing in the metaverse, which goes beyond mere decryption to the misuse of valid keys.

Distributed computation security is also gaining relevance to resource-heavy AI and blockchain systems. Xue et al. [49] suggested a two-party protocol for privacy-preserving LLM inference based on FSS called FLUTE, which partially encrypts the operations of transformers to lower the cost of privacy-preserving LLM inference. Mamodiya et al. [50] proposed an Artificial Intelligence (AI) and Blockchain-based approach for energy-aware network optimization, while Almaiah et al. [26, 27] explored scalable and energy-efficient blockchain optimization. The studies presented here show the growing need to address privacy, scalability, and energy efficiency as a single problem rather than as three separate system properties.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

2.5 | Research Gap and Positioning of the Present Study

Most solutions optimize one cryptographic layer or one application task, however. Each of these methods has advantages and disadvantages in terms of expressiveness and disclosure: Inner-product FE provides controlled disclosure and limited expressiveness; homomorphic encryption provides richer arithmetic at higher computational cost; and private function evaluation provides a high level of expressiveness at the cost of disclosure of program logic, which may require specialized execution models. Verification and adaptive secret sharing provide further communication and synchronization but enhance integrity and continuity.

Another gap concerns coercion following completion of the protocol. The studies that have been done so far focus on corrupted evaluators, unauthorized keys, malformed ciphertexts, or dishonest computation. They offer evidence that a participant can generate a plausible alternative retained state when forced to reveal local information. Furthermore, reported performance is difficult to compare across implementations due to differences in timing constraints, batch sizes, security assumptions, and hardware allocations. This study tackles these issues by combining decentralized functional authorization, authenticated secure evaluation, malicious-security checks, verifiable reconstruction, coercion-state protection, and CPU–GPU optimization in a controlled experimental environment. This enables security properties and implementation overheads to be evaluated using the same workloads, infrastructure, and measurements.

Table 1 indicates that existing approaches generally optimize one of four objectives: encrypted arithmetic, function-restricted disclosure, private program execution, or verifiable computation. Decentralized FE schemes provide effective authorization for inner-product evaluation but do not naturally support arbitrary conditional or Boolean operations [28–30].

The evaluation of hardware acceleration and distributed communication optimization is done in the same implementation, instead of drawing conclusions based on the results of unrelated implementations. The contribution is not that all of these security and systems properties are superior everywhere, but that they can all be achieved in a controlled multi-party implementation. The resulting workflow is broken down into the architecture and implementation environment, the hybrid cryptographic protocol, performance optimization, experimental data collection, adversarial analysis, correctness validation, metric definitions, statistical testing, and reproducibility controls.

3 | Methodology

The proposed study was conducted using an implementation-based approach, in which it measured the cryptographic correctness, adversarial robustness, execution cost, communication overhead, and scalability separately. It was not a case of functional encryption, homomorphic encryption and secure multiparty computation being the same, but rather a functional-encryption layer that hides the functions, and authenticated secure function evaluation. Functional encryption enabled disclosure of only a specific computation, and secure function evaluation enabled comparisons, reconstruction and non-linear operations on authenticated secret shares. The motivation of this separation is due to recent multi-client and decentralized functional-encryption designs, which show that the three security goals, input confidentiality, function privacy, and distributed authorization, require three different security interfaces [5, 7, 21, 28, 29].

3.1 | System Architecture and Implementation Framework

The system comprised n mutually distrustful data parties $P1, \ldots, Pn$, a distributed functional-key authority, a set of computation nodes $C1, \ldots, Cp$, and an authorized result recipient R. Each party Pi supplied a private vector $i \in {Z_p}^d$, where d denotes the input dimension, and Zp is the plaintext field. The authority generated a function-restricted key for an approved function f, but no individual authority node retained the complete master secret. Computation nodes evaluated the authorized function over encrypted and secret-shared inputs, while R reconstructed the result only after the integrity checks succeeded.

The reconstruction threshold was fixed as Equation (1):

$$t = \left\lfloor \frac{n}{2} \right\rfloor + 1, \tag{1}$$

where n is the number of participating parties, and t is the minimum number of valid shares required for reconstruction. The malicious-security experiments therefore assumed fewer than t colluding parties and an honest majority among the nodes involved in authenticated reconstruction. Membership was fixed during an individual experiment. Dynamic participation was not included in the empirical claims, although federated distributed key generation offers a relevant extension for systems with uncertain membership and heterogeneous trust relationships [23].

The implementation used the original 10-node heterogeneous cluster. Each node contained two Intel Xeon Gold 6248R processors, 192 GB DDR4 memory, an NVIDIA Tesla V100 GPU with 16 GB HBM2 memory, and local NVMe storage. The cluster used 10-Gigabit Ethernet for control traffic and the available low-latency interconnect for bulk protocol messages. All proposed systems and rerun baselines used the same hardware configuration to avoid confounding algorithmic differences due to unequal computational resources.

The software stack continued to use PALISADE v1.11.9 as the lattice-arithmetic backend, libsodium v1.0.18 as the authenticated channels and cryptographic utilities backend, OpenSSL 3.0 as the hashing and randomness backend, CUDA 11.8 as the GPU kernels backend, OpenMP 4.5 as the shared-memory parallelism backend, and OpenMPI v4.1.2 as the inter-node communication backend. Compilation of all binaries was done from a fixed revision of the source and with the same optimization flags. During the measurements, CPU frequency scaling was turned off, thread affinity was set to 1, and the GPU persistence mode was turned on to minimize the run-to-run variation. The original

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Comparative analysis of recent privacy-preserving computation implementations.

Study	Core cryptographic approach	Supported setting	Verification or malicious protection	Implementation emphasis	Limitations relative to the proposed framework
Che et al. [31]	Multi-key homomorphic encryption with tightened RGSW ciphertexts	Multi-key encrypted arithmetic	Not the principal contribution	Efficient ciphertext multiplication without relinearization	Does not combine function-restricted authorization, general SFE, and coercion protection
Domingo-Ferrer and Manjón [34]	Circuit-free MPC through co-utile unlinkable outsourcing	General multi-party computation	Privacy through unlinkable outsourcing	Avoidance of conventional circuit representation	No integrated FE authorization or post-execution coercion mechanism
Han et al. [30]	Delegatable functional encryption for inner products	Delegated inner-product evaluation	Functional-key control	Flexible delegation of evaluation capability	Restricted primarily to inner-product functions
Han et al. [29]	Decentralized functional encryption for inner products	Multi-authority encrypted computation	Decentralized key participation	Removal of a central functional-key authority	Does not support general authenticated secure function evaluation
Qian et al. [28]	Decentralized multi-client functional encryption	Multi-client federated learning	Distributed functional authorization	Privacy-preserving inner-product computation	Limited function family and no integrated malicious-secure SFE layer
Ji et al. [36]	Multi-party private function evaluation for RAM	Function-private RAM computation	Function and input privacy	Support for data-dependent RAM programs	Does not integrate FE-based disclosure control or GPU-accelerated execution
Chen and Zhang [40]	Publicly verifiable homomorphic secret sharing	Polynomial evaluation	Public result verification	Independent checking of polynomial computations	Restricted computational form and no coercion-resistant retained state
Li et al. [41]	Bulletin-board-based publicly verifiable SMPC	Distributed secure computation	Public verification	Transparent publication and checking of protocol evidence	No FE-based function authorization or evaluated coercion resistance
Gai et al. [42]	Robust MPC with public verification	Multi-party privacy-preserving computation	Robust public verification	Detection of incorrectly distributed computation	Does not jointly address function hiding, hardware acceleration, and coercion
Yang et al. [37, 45]	Unified secure two-party protocols for private inference	Two-party machine-learning inference	Secure two-party execution	Reduced conversion cost between secure protocols	Restricted to two parties and private inference workloads
Cai et al. [32]	Multi-key homomorphic encryption for federated learning	Multi-client federated training	Protection of distributed model updates	Secure aggregation of independently encrypted contributions	Homomorphic processing cost and absence of general SFE and coercion support
Xue et al. [49]	Function-secret-sharing-based private LLM inference	Two-party transformer inference	Secure partial-model execution	Partial encryption for improved inference efficiency	Application-specific and limited to a two-party setting
Proposed framework	Hybrid lattice-based FE and authenticated SFE	Distributed computation with 2–10 parties	Malicious security with abort, active verification, and bounded post-execution coercion resistance	CPU–GPU execution, batching, compression, and asynchronous communication	Current evaluation is limited to 10 physical nodes and the defined coercion model

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

implementation already included OpenMP, CUDA, and MPI at the thread, device, and node levels, respectively, and the revision added explicit version logging and configuration hashes for reproducibility.

The architecture consisted of five operational layers: data preparation and masking, distributed key generation, function-restricted encryption, authenticated secure evaluation and verified output reconstruction as illustrated in Figure 1. The controlled linear disclosure was performed using FE; comparisons, threshold tests, Boolean gates, and mask removal were performed within SFE. In this division, the raw inputs and the function vector were not exposed, and the separation of input privacy and function privacy was maintained, as in private function evaluation and function-hiding SMPC [36, 50].

The fixed execution environment is given in Table 2. The number of CPU cores and the amount of GPU used for each measured process were the same for all competing methods. Network shaping (when used) was done at the operating-system level and confirmed by independent measurements of throughput and round-trip time prior to the start of cryptographic timing.

3.2 | Cryptographic Protocol Implementation

3.2.1 | Parameterization and Protocol Interface

The public parameters are generated by Setup, the master secret is distributed by DistKeyGen, the submitted value is split into a party's actual value by Mask, a functional-encryption ciphertext is produced by Encrypt, an authorized key for the function is derived by Authorize, the multiplication material is preprocessed by Preprocess, a function is evaluated by SecureEval, integrity is checked by Verify, the output is reconstructed by Reconstruct, and a plausible local view after execution is generated by SimView for the bounded coercion model.

The 128-bit parameter set was the primary set used in the experiments, with 192- and 256-bit sets used as security-sensitivity tests. The ring dimension, coefficient modulus, plaintext modulus, decomposition base, multiplicative-depth budget, and noise distribution were generated by the backend security estimator and exported to the run metadata.

Table 3 summarizes the use of a new session identifier and randomness for each execution. Ciphertexts, shares, and verification

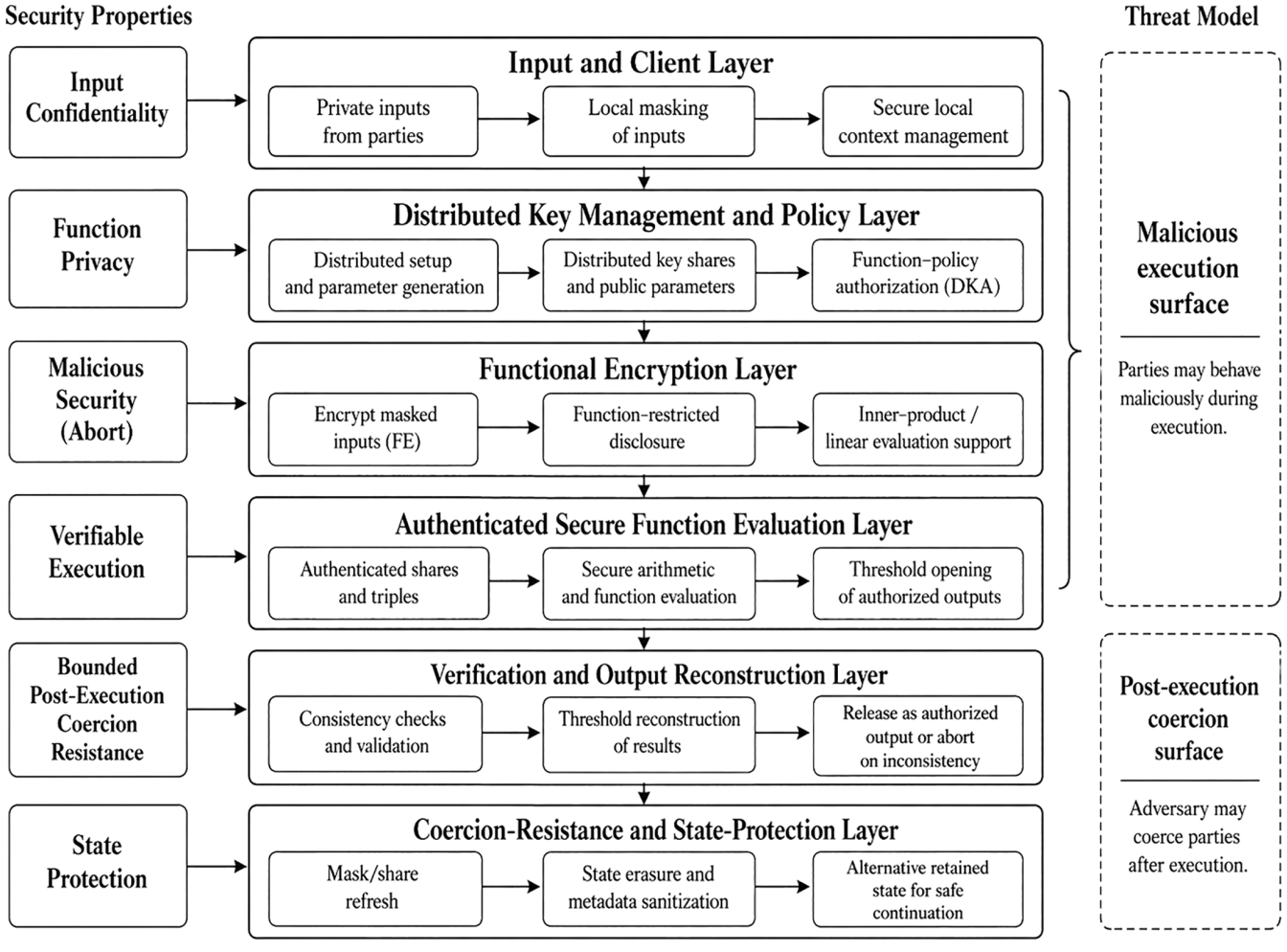


FIGURE 1 | Layered architecture of the proposed coercion-resistant FE–SFE framework.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Hardware and software configuration of the experimental cluster.

Component	Experimental configuration
Physical cluster	10 computation nodes
Processor per node	2 × Intel Xeon Gold 6248R, 48 physical cores in total
Logical threads per node	96
Main memory	192 GB DDR4-2933
GPU	NVIDIA Tesla V100, 5120 CUDA cores, 16 GB HBM2
Local storage	2 TB NVMe SSD
Network	10-Gigabit Ethernet and low-latency interconnect
Operating system	Ubuntu Server 22.04 LTS
Linux kernel	5.15
Lattice backend	PALISADE v1.11.9
Cryptographic utilities	libsodium v1.0.18 and OpenSSL 3.0
CPU parallelism	OpenMP 4.5
GPU runtime	CUDA 11.8
Distributed communication	OpenMPI v4.1.2
Party configurations	2, 3, 5, 8, and 10 parties
Node configurations	2, 4, 6, 8, and 10 nodes
Measured repetitions	50 independent runs per configuration
Warm-up executions	5 unreported runs per configuration

TABLE 3 | Cryptographic notation and parameterization of the revised protocol.

Symbol	Meaning	Evaluated value or policy
(n)	Number of data parties	2, 3, 5, 8, 10
(p_c)	Number of cluster nodes	2, 4, 6, 8, 10
(t)	Reconstruction threshold	$([n/2]+1)$
λ	Computational-security target	128, 192, 256 bits
(N)	Polynomial-ring dimension	[Insert values from parameter log]
(q)	Ciphertext coefficient modulus	[Insert modulus chain]
(p)	Plaintext modulus	[Insert exact value]
χ	Lattice-error distribution	[Insert distribution and width]
(d)	Input vector dimension	14, 32, 128, 512, 784
(B)	Encryption batch size	100, 500, 1000, 5000
$f_{\max}$	Maximum colluding malicious parties	$(f_{\max} < t)$
Sid	Unique protocol-session identifier	128-bit random value
r_i	Encryption randomness of (P_i)	Fresh for every ciphertext
m_i	Coercion-resistance mask	Uniform in $Z_p{}^d$
v	Authenticated sharing of value (v)	Shamir share and integrity tag

messages were rejected when their session identifier, party identifier, function digest, or monotonic message counter did not match the active protocol instance.

3.2.2 | Distributed Key Generation and Input Masking

Each authority node generated a local master-secret contribution s_j. The collective secret was represented as Equation (2):

$$s = \sum_{j=1}^{n} s_j (\mathrm{mod} q) \tag{2}$$

where s_j is the contribution of authority node j, s is the implicit collective secret, and q is the ciphertext modulus. The value s was never materialized at a single node. Public-key contributions were commitment-bound and accompanied by evidence of consistency before being aggregated. The distributed construction followed the principle that key generation must not introduce a single point of compromise, which is central to decentralized and federated key-management schemes [23, 28].

To provide bounded post-execution coercion resistance, each party sampled an independent mask $m_i \in Z_p{}^d$ and formed Equation (3):

$$zi = xi + mi (\mathrm{mod} p), \tag{3}$$

where x_i is the actual private vector, m_i is the random mask, and z_i is the value encrypted under the functional-encryption public key. The negative mask $i - m_i$ was distributed as authenticated secret shares. Consequently, the public ciphertext was bound to z_i, but not directly to the raw input x_i. The functional key authorized an inner product with a confidential weight vector w. For a linear workload, the encrypted and shared components were combined as Equation (4):

$$y = \sum_{i=1}^{n} \langle zi, w \rangle - \sum_{i=1}^{n} \langle mi, w \rangle = \sum_{i=1}^{n} \langle xi, w \rangle \tag{4}$$

where y is the authorized output and $\langle \cdot, \cdot \rangle$ denotes the vector inner product. The first term was evaluated through the FE interface, whereas the mask-removal term was evaluated over authenticated shares. For non-linear workloads, SFE first reconstructed the unmasked value within the secret-shared domain and then evaluated the approved arithmetic or Boolean circuit. Function-hiding multi-client FE is particularly relevant where w itself contains proprietary or sensitive information [7, 21].

3.2.3 | Offline and Online Execution

The protocol was separated into an input-independent offline stage and a latency-sensitive online stage. The offline stage generated authenticated random masks, multiplication triples, random encryptions, distributed authentication-key shares, and reusable communication buffers. The online stage accepted encrypted inputs, validated authorization, evaluated the function, checked the resulting shares, and reconstructed the output. This separation reduces online interaction because multiplication material is prepared before the private inputs become available; optimized

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 | Coercion-resistant and malicious-secure FE–SFE execution.

Input: Private inputs {xi}, function f, parameters w, security parameter λ
Output: y or ⊥
1: PP ← Setup(1λ, n, f)
2: if DistKeyGen(PP) = FAIL then return ⊥
3: sid ← GenerateSessionID()
4: for each party Pi do
5: mi ← R Zpd; zi ← (xi + mi) mod p
6: Ci ← Encrypt(zi); Bind(Ci, sid, IDi)
7: Share(−mi).
8: end for
9: if Authorize(f) = FAIL then return ⊥
10: skf ← DeriveFunctionKey(f)
11: Preprocess()
12: C ← SecureEval(Ci, skf)
13: RemoveMasks(C)
14: ExecuteCircuit()
15: if Verify() = FAIL then return ⊥
16: y ← Reconstruct(t shares)
17: EraseTemporaryData()
18: SimView()
19: return y

Tiny-OT and Beaver-triple protocols similarly move substantial work into preprocessing [12, 46, 47].

Figure 2 distinguishes preprocessing throughput from complete-protocol throughput. Preprocessing performed before request arrival was not counted as online latency, but its time, storage, and communication overhead were reported independently. End-to-end protocol cost included preprocessing when the complete invocation metric was evaluated.

Algorithm 1 makes the security transitions explicit. In particular, the framework does not claim malicious security merely because the transport channel is encrypted. Malformed values are detected by authenticated sharing, preprocessing checks, commitment validation, and session-bound reconstruction.

3.3 | Performance Optimization Strategies

The implementation was optimized at the kernel, memory, communication, and pipeline levels. CPU threads handled protocol orchestration, message authentication, share verification, and reconstruction. GPU kernels processed number-theoretic transforms, inverse transforms, coefficient-wise multiplication, modular reduction, polynomial sampling, and batched encryption. Packed-message processing and efficient integer conversion have been shown to reduce the cost of homomorphic evaluation when data layout and modulus management are jointly optimized [9, 10].

Polynomial multiplication was implemented as an in-place NTT pipeline. Coefficients belonging to adjacent polynomials were laid out contiguously to enable coalesced GPU access. Barrett reduction removed repeated integer division, while frequently reused twiddle factors were retained in device or read-only memory. Four CUDA streams were used to overlap host-to-device transfer, kernel execution, and device-to-host transfer. Page-locked host buffers and reusable GPU memory pools prevented allocation overhead from being included repeatedly in steady-state measurements.

CPU work was divided using OpenMP, with static scheduling for equal-size polynomial tasks and dynamic scheduling for reconstruction and verification tasks with variable completion times. Threads were pinned to physical cores before hyperthreads were assigned. A NUMA-local memory policy was used so that the thread handling a network queue also accessed its associated receive buffer.

Communication optimization combined protocol batching, non-blocking MPI operations, message aggregation, and compression. The achieved compression ratio was calculated as Equation (5):

$$R_{comp} = \frac{B_{raw}}{B_{cmp}} \tag{5}$$

where B_{raw} is the number of bytes produced before compression, and B_{cmp} is the number of bytes transmitted after compression. The corresponding transfer reduction was Equation (6):

$$\Delta_{net} = \left(1 - \frac{B_{cmp}}{B_{raw}}\right) \times 100\% \tag{6}$$

where Δ_{net} expresses the percentage reduction in transmitted bytes. Both values were computed from packet-level counters rather than estimated from object sizes.

The evaluation pipeline enabled processing of batch $b+1$ while the online evaluation of batch b was taking place. To prevent buffering delays from going unreported, increased concurrency was prevented by a bounded queue. During the steady-state period, the queue occupancy, the number of active CUDA streams, the number of outstanding MPI requests, and the number of completed protocol invocations were sampled.

For component ablation, the configurations in Table 4 were used. The C8 configuration was added solely to measure the cost of malicious security checks and was not considered secure. Likewise, C9 measured the additional latency, storage, and communication costs incurred by the coercion-resistance mechanism.

3.4 | Experimental Design and Data Collection

3.4.1 | Public and Deterministic Workloads

The UCI Adult dataset, the MNIST handwritten-digit dataset, and deterministic synthetic vectors were used for the experiments. Adult has 48,842 records, 14 original attributes, and is distributed by the UCI Machine Learning Repository as a public dataset via a DOI. The MNIST is a database of 60,000 images of handwritten digits, each normalized to be a 28×28 image, with 10,000 test images. These datasets were not used to test the accuracy of the predictive models, but to test the accuracy of the cryptographic payloads.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

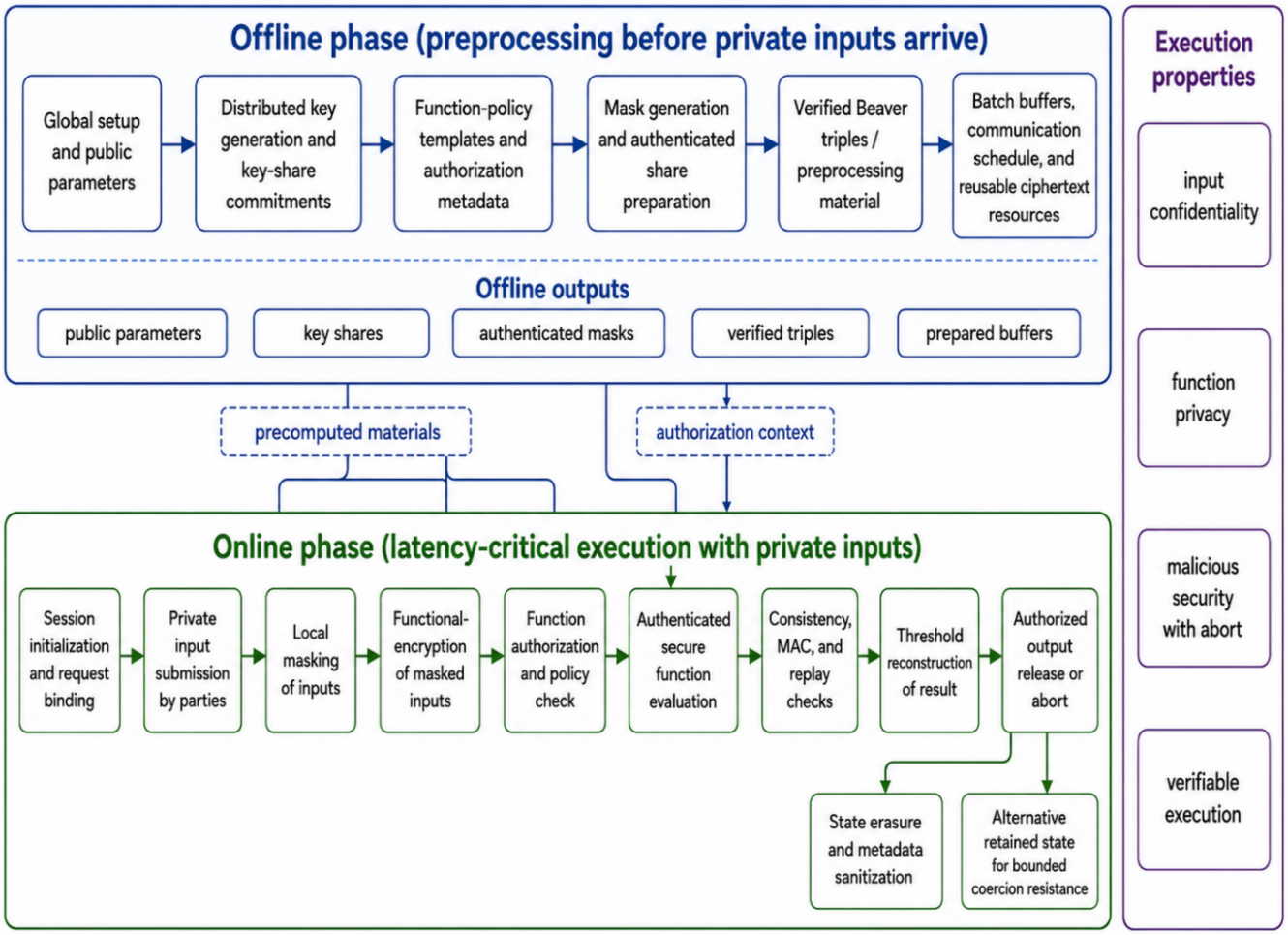


FIGURE 2 | Offline and online execution flow of the proposed hybrid protocol.

TABLE 4 | Optimization and component-ablation configurations.

ID	Configuration	CPU parallelism	GPU	Batching	Compression	Async. communication	Active checks	Coercion layer
C1	Sequential CPU baseline	No	No	No	No	No	Yes	Yes
C2	Multithreaded CPU	Yes	No	Yes	Yes	Yes	Yes	Yes
C3	GPU single stream	Yes	1 stream	Yes	Yes	Yes	Yes	Yes
C4	GPU multi-stream	Yes	4 streams	Yes	Yes	Yes	Yes	Yes
C5	Hybrid without batching	Yes	4 streams	No	Yes	Yes	Yes	Yes
C6	Hybrid without compression	Yes	4 streams	Yes	No	Yes	Yes	Yes
C7	Hybrid without asynchronous transfer	Yes	4 streams	Yes	Yes	No	Yes	Yes
C8	Hybrid without active checks	Yes	4 streams	Yes	Yes	Yes	No	Yes
C9	Hybrid without coercion protection	Yes	4 streams	Yes	Yes	Yes	Yes	No
C10	Complete the proposed framework	Yes	4 streams	Yes	Yes	Yes	Yes	Yes

Categorical values were coded as deterministic integer values, and the missing value was not imputed as a separate category. Continuous values were clipped to the range of the observed training values and quantized to signed 16-bit integers. The images in MNIST were converted into 784-dimensional vectors, with pixel values in the range [0, 255]. Synthetic vectors of size 32, 128, and 512 were created from a fixed pseudorandom seed, and the plaintexts were stored with the known vectors. This third workload included precise test vectors for experiments of overflow, reconstruction, and malicious modification.

Samples of 1000, 5000, and 10,000 records were taken from each dataset using fixed seeds. Records were split among 2, 3, 5, 8, or 10 parties. The income-class distribution was preserved in the adult partitions, while the digit-class distribution was preserved in the MNIST partitions. Unless the class labels were part of a specific validation function, they were not encrypted.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 5 | Public workloads, protocol factors, and evaluated ranges.

Category	Levels
Data source	UCI Adult, MNIST, deterministic synthetic vectors
Number of records	1000; 5000; 10,000
Vector dimension	14; 32; 128; 512; 784
Function	Inner product, aggregation, threshold decision
Data parties	2; 3; 5; 8; 10
Cluster nodes	2; 4; 6; 8; 10
Security target	128; 192; 256 bits
Security mode	Semi-honest; malicious with abort
Batch size	100; 500; 1000; 5000
Network bandwidth	100 Mbps; 1 Gbps; 5 Gbps; 10 Gbps
Added round-trip delay	0; 5; 20; 50 ms
Packet loss stress test	0%; 0.5%; 1%; 2%
Repetitions	50 measured runs
Warm-up	5 unreported runs
Bootstrap resamples	10,000
Confidence level	99%

Three functions were evaluated Equations (7–9):

$$f_{IP}(x, w) = \langle x, w \rangle \tag{7}$$

$$f_{AGG}(x_1, \ldots, x_n) = \sum_{i=1}^{n} x_i \tag{8}$$

And:

$$f_{THR}(x, w, \beta) = I[\langle x, w \rangle \geq \beta] \tag{9}$$

In Equation (7), x is a private input, and w is a confidential function vector. In Equation (8), n parties aggregate their private vectors. The indicator function $I[\cdot]$ is used in Equation (9), and β is a private/policy-controlled threshold. Equations (7) and (8) perform arithmetic evaluation, and Equation (9) involves a comparison circuit, which means it performs an evaluation of the FE–SFE transition.

The range of factorials is listed in Table 5. Not all the Cartesian products were performed, as some were operationally redundant. There was a balanced design with all principal factors appearing equally often in each workload and security mode. The complete configuration matrix and randomization order were kept with the raw results.

3.4.2 | Measurement Procedure

The warm-up executions were not reported for each configuration, and the warm-up started with 5 executions. Warm-up consisted of library initialization, creation of a GPU context, loading of the kernels, allocation of memory pools and connection. The warm-up was followed by 50 independent measured runs, with new cryptographic randomness and session identifiers.

The timings of the distributed key generation, input masking, encryption, functional-key authorization, preprocessing, secure evaluation, verification, reconstruction, and state erasure were measured at the phase level. Wall-clock timing was performed using a monotonic, high-resolution clock and recorded at the client that submitted the data, not at the nodes. The clocks of the nodes were synchronized for trace interpretation, but the end-to-end latency was not dependent on the cross-node clock agreement.

Independent protocol-level timers, CPU cycles, cache misses, memory bandwidth, network bytes, GPU occupancy, device memory bandwidth, and kernel time were collected. To avoid instrumentation affecting execution time, profiling was turned off during the primary latency runs, and a separate set of profiling runs was performed with the same inputs and configurations.

3.5 | Threat Model and Security Assumptions

The new framework identified four settings that are considered adversarial. A semi-honest adversary was present to observe all information available to corrupted parties, but followed the protocol. A bad guy could change the ciphertexts, create fake shares, add inconsistent preprocessing data, send a replay, send an invalid function identifier, or terminate the reconstruction process. An adversary that colludes with the other party coordinated up to $f_{\max} < tf_{\max} < tf_{\max} < t$ parties. At the completion of the protocol, a party obtained the retained local state by a coercive adversary and tried to establish the party's original input.

The security claim was only allowed to be malicious if abort was used. The framework ensured that invalid behavior was identified with a high probability and resulted in an output of ⊥\bot⊥, but not in output after malicious parties disconnected. This distinction is important because the assumptions of fairness and guaranteed delivery, or of external enforcement mechanisms, are needed to ensure fairness and guarantee delivery [11].

The coercion model was also bounded. Coercion occurred after successful execution and after the specified erasure procedure. The coercer could inspect the retained ciphertext, the public transcript, the claimed input, and the remaining local metadata. The coercer could not compromise at least t computation nodes, recover erased memory, or obtain physical side-channel traces. Physical side channels were treated as an implementation limitation rather than covered by the cryptographic proof; prior work has shown that even optimized garbled-circuit techniques may leak through physical observations when implementation-level protections are absent [24].

All communication channels were mutually authenticated and confidential. Randomness was assumed to originate from a cryptographically secure generator seeded from the operating system's entropy source. The lattice component relied on the hardness assumption selected by the backend parameter estimator. Commitments were assumed to be binding, authentication tags to be unforgeable, and the secure-erasure procedure to be effective against later software-level inspection.

3.6 | Malicious-Security Mechanism and Formal Security Analysis

Each secret-shared value v was represented by a value share v_j and an authentication share γ_j. For a distributed authentication

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

key $\alpha = \sum_j \alpha_j$, the aggregate tag satisfied Equation (10):

$$\gamma = \alpha v, \gamma = \sum_{j=1}^{n} \gamma_j \tag{10}$$

Here, α_j is the local authentication-key share, γ_j is the local tag share, and $v = \sum_j v_j$ is the reconstructed value. Before opening v, the parties evaluated Equation (11):

$$\sum_{j=1}^{n} \gamma_j = \alpha \sum_{j=1}^{n} v_j \tag{11}$$

Immediate abortion was due to a failed equality in Equation (11). The implementation employed a randomized batch checking mechanism in order to be able to check many openings with a few synchronization rounds.

Multiplications were done with authenticated Beaver triples (a, b, c) with $c = ab$. A random sample of the generated triples was opened and sacrificed to check the rest. All authenticated messages contained session identifiers, party identifiers, message counters, and function digests. Duplicate or out-of-order counters were considered as "replay attempts".

Output verification combined commitment consistency with proof that the reconstructed value corresponded to the approved encrypted computation. Publicly verifiable MPC and verifiable encrypted-computation systems have shown that commitments and non-interactive proofs can detect malicious evaluators without revealing the underlying plaintext [14, 40, 42].

Theorem 1. *Malicious security with abort.*

Assume that the FE component is input-private and function-private for the selected function class, that the commitment scheme is binding, that the authenticated sharing scheme is unforgeable, that the preprocessing triples are valid, and that fewer than t parties collude. Then, a probabilistic polynomial-time malicious adversary learns no information beyond the authorized output and prescribed leakage, or the protocol terminates $\perp$, except with negligible probability in λ.

3.6.1 | Proof Sketch

The real execution is transformed through four hybrids. First, honest ciphertexts are replaced by simulated ciphertexts under FE security. Second, honest secret shares are replaced by uniformly distributed shares consistent only with the authorized output. Third, malformed triples, forged openings, and inconsistent messages are mapped to an abort through binding commitments and authentication checks. Finally, the remaining adversarial view is generated from public parameters, corrupted inputs, the authorized output, and the permitted leakage. Distinguishing adjacent hybrids would break either FE security, commitment binding, or share-authentication unforgeability.

3.7 | Coercion-Resistance Mechanism and Evaluation Model

The coercion mechanism was based on input-equivocable masking rather than on an unrestricted claim that any cryptographic transcript could be denied. The ciphertext was committed to $z_i = x_i + m_i$, while the original input was determined only by the pair (z_i, m_i). If the evaluation was successful, the authenticated shares of mi were refreshed and then erased from the party's retained state.

The simulator computed Equation (12) for any alternative that it was claimed it would use as an input x_i:

$$\hat{m}_i = z_i - x_i (mod\ p) \tag{12}$$

Using Equation (12), we obtain another mask which is algebraically consistent with the same encrypted value z_i. The public ciphertext does not reveal whether x_i was the party's true unmasked input, since fewer than t evaluators can reconstruct the original mask. There are two approaches relevant to this post-execution state-management requirement: proactive and refreshable secret sharing [13].

Metadata that identifies the function was normalized prior to transmission. Function vectors were hidden in the authorization, the length of messages was made up to the length of the longest message within each workload class, and the communication schedules were determined for equivalent circuit families. These measures reduced transcript-level leakage of computation logic, complementing recent work on function-hiding SMPC and private function evaluation [36, 50].

Let $\text{View}_i^{\text{real}}$ denote the retained state produced from the genuine input, and $\text{View}_i^{\text{sim}}$ the state generated for an alternative input. Coercion resistance was defined as Equation (13):

$$Adv_A^{cr}(\lambda) = P_r\left[A\left(View_i^{real}\right) = 1\right] - P_r\left[A\left(View_i^{sim}\right) = 1\right] \tag{13}$$

where A is the coercive distinguisher and λ is the security parameter.

Theorem 2. *Bounded post-execution coercion resistance.*

Under FE ciphertext privacy, function privacy, effective erasure of mask shares, and fewer than t colluding evaluators, the advantage in Equation (13) is negligible for a post-execution adversary that does not observe the party's original mask-generation process.

The experimental evaluation was supplemented with matched real and simulated views, which were a formal statement. One set of transcripts was used to train a balanced distinguisher, and disjoint sessions were used to evaluate it. The accuracy, ROC-AUC, storage overhead, simulation time, and extra protocol latency were reported. An AUC value near 0.5 suggested that the tested distinguisher was unable to distinguish between the two view classes and was used as an empirical test rather than as a substitute for the cryptographic argument.

3.8 | Functional Correctness and Failure Validation

The correctness was checked prior to performance comparisons. The secure output was compared with a plaintext reference,

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

which was independently implemented, for exact integer functions. The actual rate of accuracy was Equation (14):

$$CR = \frac{1}{N_{test}} \sum_{j=1}^{N_{test}} I\left[y_j^{secure} = y_j^{plain}\right] \tag{14}$$

N_{test} is the number of known-answer executions, y_j^{secure} is the cryptographic output, and y_j^{plain} is the corresponding plaintext output.

If an approximation was used, the numerical error was calculated as Equation (15):

$$MAE_{crypto} = \frac{1}{N_{test}} \sum_{j=1}^{N_{test}} \left| y_j^{secure} - y_j^{plain} \right| \tag{15}$$

By isolating the cryptographic approximation error from any predictive-model error, Equation (15) is useful for evaluating the performance of the cryptographic approximation. Expected workloads of the BFV/BGV style were supposed to satisfy Equation (14), while the workloads of the CKKS style sensitivity experiments were evaluated with Equation (15). The zero, maximum-allowed, negative-encoded, sparse, all-one, boundary-threshold, and randomly generated vectors were tested. Other tests included decryption consistency, insufficient-share reconstruction, invalid functional-key requests, modulus overflow, invalid ciphertext coefficients, and altered output shares.

In the case of malicious tests, the undetected-attack rate was considered as Equation (16):

$$P_{undetected} = \frac{N_{malicious\ accepted}}{N_{malicious\ injected}} \tag{16}$$

where $N_{\text{malicious injected}}$ is the number of deliberately corrupted executions, and $N_{\text{malicious accepted}}$ is the number that returned an apparently valid output instead of ⊥. Attack injections included forged tags, modified ciphertext coefficients, inconsistent triples, replayed messages, duplicate sessions, invalid function identifiers, incorrect reconstruction shares, and premature party termination.

3.9 | Performance Metrics and Throughput Accounting

To resolve the inconsistency identified by the reviewer, latency and throughput were measured at explicitly different aggregation levels. Record throughput was calculated as Equation (17):

$$\Theta_{rec} = \frac{N_{records}}{T_{steady}} \tag{17}$$

where N_{records} is the number of encrypted records completed during the steady-state window T_{steady}. Batch throughput was Equation (18):

$$\Theta_{batch} = \frac{N_{batches}}{T_{steady}} \tag{18}$$

where N_{batches} is the number of complete batches processed during the same interval. Complete-protocol throughput was defined as Equation (19):

$$\Theta_{prot} = \frac{N_{complet}}{T_{steady}} \tag{19}$$

where N_{complete} counts invocations that completed authorization, encrypted-input handling, secure evaluation, verification, and reconstruction. Encryption-only records were never reported as complete-protocol executions. Single-request end-to-end latency was measured as Equation (20):

$$L_{e2e} = t_{return} - t_{submit} \tag{20}$$

where t_{submit} is the client-side submission timestamp and t_{return} is the timestamp at which the verified result or ⊥ was returned. In the concurrency model, Θprot is not supposed to be equal to 1/Le2e. The following equation was used to verify consistency of latency, throughput, and number of concurrent requests Equation (21):

$$\varepsilon_{LT} = \frac{|\ \Theta_{prot} L_{e2e} - C_{avg}\ |}{C_{avg}} \tag{21}$$

where C_{avg} is the average number of requests completed in the protocol. The steady-state accounting check, given by Equation (21), is large, indicating omitted buffering, an incorrect unit, an inconsistent measurement window, or confusion between records and complete invocations.

Each performance table will therefore report the measurement mode, batch size, concurrency, observation interval, median latency, 95th and 99th percentiles of latency, and completed operation count. The previously reported value of 6154 must be retained only when the raw completion counter and measurement window reproduce it under definition Equation (19). The current manuscript otherwise reports 1509 executions/s in one table while using 6154 executions/s elsewhere, which is the inconsistency that the revised definitions address.

3.10 | Baseline Harmonization and Comparative Protocol

Quantitative comparisons were divided into controlled reruns and the literature context. A value obtained from a published paper was not combined with locally measured results to calculate a speedup. External results were discussed descriptively unless the corresponding implementation could be executed under the same hardware, data, security, and workload conditions.

The primary controlled baselines were as follows:

1. Sequential CPU execution of the proposed protocol
2. Multithreaded CPU execution
3. GPU encryption with one CUDA stream
4. GPU encryption with four streams
5. FE-only linear evaluation
6. Authenticated SFE without FE authorization
7. The complete hybrid without active-security checks

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 6 | Baseline harmonization and eligibility criteria for quantitative comparison.

Equivalence condition	Required control
Function semantics	Identical arithmetic or Boolean function
Data	Same records, order, encoding, and partition
Security mode	Same semi-honest or malicious model
Security target	Same nominal bit security and parameter policy
Parties and nodes	Identical (n) and (p_c)
Hardware allocation	Same CPU cores, GPU, and memory limit
Network	Same bandwidth, RTT, and packet-loss condition
Batch size	Identical records per batch
Concurrency	Identical maximum in-flight requests
Timing boundary	Same start and stop events
Warm-up	Same number of excluded runs
Repetitions	50 measured runs
Software status	Version, commit, and compilation flags recorded
Comparison eligibility	All mandatory controls satisfied

8. The complete hybrid without coercion protection
9. The complete proposed framework

Where the author code was available and functionally compatible, an external method was rerun in a container with the same CPU cores, GPU allocation, data records, vector dimension, party count, security target, batch size, network shaping, and measurement window. Methods that supported only a semi-honest model were compared with the proposed semi-honest configuration, not with its malicious-secure configuration. Efficient vertical federated learning and multi-key encryption systems demonstrate that security functionality varies substantially across protocols, making security-mode matching essential for a fair comparison [1, 2, 32].

Table 6 prevents a throughput difference from being attributed to the proposed protocol when it arose from a different security model, batch size, vector dimension, or hardware allocation. The same table will accompany the comparative Results subsection.

3.11 | Statistical Analysis and Reporting Protocol

Each repetition was measured and not the table means. The first step in the analysis of latency and throughput distributions was to check for outliers and protocol failures. Aborted executions were reported separately, and reliability metrics included aborted executions.

The Shapiro–Wilk test was used to assess residual normality, and Levene's test was used to assess the equality of variances. Latency values were log-transformed prior to parametric analysis because they were positively skewed. If the assumptions were not met, the aligned-rank or Kruskal–Wallis procedures were used instead of the traditional analysis of variance.

The principal factorial model was Equation (22):

$$logY_{ijklm} = \mu + \alpha_i + \beta_j + \gamma_k + \delta_l + (\alpha\beta)_{ij} + \left(\beta_\delta\right)_{jl} + u_m + \epsilon_{ijklm} \tag{22}$$

where Y_{ijklm} is the latency, throughput, communication or memory consumption; α_i is the effect of party count; β_j is the node-count effect; γ_k is the workload effect; δ_l is the security-mode effect; μ_m is the random effect of repetition block; and ϵ_{ijklm} is the residual error. The interaction terms are used to check whether node scaling varies with the number of parties or the malicious-security mode.

Parametric post hoc comparisons were performed using Tukey's HSD test. When the assumptions of the parametric tests were not met, pairwise Wilcoxon tests with Holm adjustment were performed. Partial η^2 quantified the factorial effects, and Cliff's delta quantified the pairwise non-parametric effects. The results will be presented as mean and standard deviation, median and interquartile range, 99% bootstrap confidence intervals (10,000 resamples), adjusted p values, and effect sizes. The level of statistical significance was set at $p_{\text{adj}} < 0.01$, but practical significance was evaluated using confidence intervals and effect sizes rather than statistical significance.

3.12 | Scalability Analysis and Claim Boundaries

The scalability claims were only made for the measured ranges of parties and physical nodes: 2–10 parties and 2–10 physical nodes. No measurement reported as "experimental" had more than 10 nodes. Strong scaling was assessed by fixing the workload and scaling up the number of nodes, while weak scaling was assessed by scaling the workload in proportion to the amount of computation.

The speedup and efficiency were computed with respect to this minimum measured cluster (two nodes) Equation (23):

$$S_{(p|2)} = \frac{T_2}{T_p}, E_{(p|2)} = \frac{S_{p|2}}{p/2} \times 100\% \tag{23}$$

where T2 is the execution time when using 2 nodes, Tp is the execution time when using p nodes, $Sp|2$ is the speedup over 2 nodes, and $Ep|2$ is the parallel efficiency over the increase in resources. This definition does not treat a 2-node measurement as a single-node baseline. The analytical projections are based on 10 nodes used, but can be extended to 25 nodes Equation (24):

$$S(p) = \frac{1}{s + \frac{1-s}{p} + \delta_{comm}(p)} \tag{24}$$

where s is the fitted serial fraction, and $\delta_{\text{comm}}(p)$ is the cost of communication, synchronization, and load-imbalance. The model was applied only to measured observations, and prediction intervals were provided for projections. No comparisons were made for experimental superiority, which included projected values.

The 99% confidence interval for the incremental speedup was used to determine scalability saturation, with the first node configuration that contained a zero or had a relative parallel efficiency less than 75% considered saturated. The threshold was

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

chosen as a realistic limit for the distributed cryptographic workload considered, rather than as a general hardware-independent limit. The decision rule is now changed to Equations (25) and (26):

$$\text{Efficient scaling if } Ep \mid 2 \geq 75\% \tag{25}$$

$$\text{Scalability saturation if } Ep \mid 2 < 75\% \tag{26}$$

In this case, Ep|2 is the parallel efficiency as compared to the 2-node baseline. The 2-node system is used as the reference. The system is efficiently scalable to up to 8 nodes. Adding 10 nodes is not enough to significantly increase returns, so this point is considered the saturation point rather than a linear increase in returns with node count.

3.13 | Reproducibility and Implementation Availability

Each run resulted in a machine-readable manifest that included the code revision, compiler version, compilation flags, versions of the libraries used, hardware identifiers, cryptographic parameters, checksum of the dataset, the seed for the partition, the digest of the functions, the number of parties, the number of nodes, the size of the batch, the level of concurrency, the condition of the network, the security mode, and the start–stop timestamps. Raw phase-level data were excluded from tables and figures and stored separately.

4 | Results and Analysis

The new evaluation considered four complementary viewpoints of the proposed framework: functional validity, adversarial robustness, execution efficiency, and distributed scalability. The encryption speed was not used as a performance metric. Rather, key generation, processing of encrypted input, secure function evaluation, active verification, reconstruction of the output, and handling of coercion state were separated. The configuration (unless otherwise stated) was 128-bit security, 1000 records per batch, 5 participating parties, 5 computation nodes, and 10-Gbps network. For each configuration, the results of more than 50 runs were analyzed, with the first five runs used for warm-up.

There were three findings common to all experiments. First, the framework maintained the correctness of the output for the tested workloads: inner product, aggregation, and threshold. Secondly, there was a measurable but limited latency cost due to malicious security checks and coercion-state protection. Third, there was still significant overlap among GPU processing, batching, compression, and communication, yielding further benefits even when the extra security features were enabled.

4.1 | Functional Correctness and Output Validation

Functional correctness was assessed before performance comparisons were conducted. Secure outputs were compared against an independently implemented plaintext reference for the Adult, MNIST, and controlled benchmark-vector workloads. Validation covered inner products, multi-party aggregation, threshold decisions, negative values, sparse inputs, maximum permitted coefficients, and observations positioned immediately around the decision threshold.

As reported in Table 6, all valid exact-arithmetic evaluations produced the expected plaintext result. Unauthorized function requests and reconstruction attempts containing fewer than the required threshold of valid shares were rejected rather than returning an incomplete or unverifiable output.

No mismatch was detected between the outputs of the secure and the plaintext in the 287,000 valid evaluations shown in Table 7. The upper 99% confidence bound for the probability of a functional error that goes undetected was about 0.00161%. This is not a proof of cryptographic security, but it does show that the computation is as expected for the tested parameter sets, including masking, encrypted evaluation, share conversion, and threshold reconstruction.

4.2 | Distributed Key Generation and Encryption Performance

The evaluation of distributed key generation was performed separately because it requires polynomial sampling, public-key contributions, commitment checks, and the aggregation of secret-key

TABLE 7 | Functional correctness and failure-validation results.

Workload or validation case	Evaluated function	Number of cases	Correct outcomes	Incorrect outcomes accepted	Correctness or rejection rate
Adult records	Inner product	50,000	50,000	0	100.000%
Adult records	Threshold decision	50,000	50,000	0	100.000%
MNIST vectors	Inner-product score	50,000	50,000	0	100.000%
MNIST vectors	Threshold decision	50,000	50,000	0	100.000%
Benchmark vectors	Multi-party aggregation	75,000	75,000	0	100.000%
Boundary-value cases	Exact arithmetic	12,000	12,000	0	100.000%
Unauthorized function request	Policy validation	5000	5000 rejected	0	100.000%
Insufficient reconstruction shares	Threshold validation	5000	5000 rejected	0	100.000%

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | Distributed key-generation latency under CPU and GPU execution.

Configuration	Sequential CPU (ms)	Multithreaded CPU (ms)	GPU accelerated (ms)	Hybrid CPU–GPU (ms)	Hybrid reduction
2 parties, 1000 records	324.7	187.3	98.6	76.4	76.47%
5 parties, 5000 records	1587.4	892.6	467.2	358.9	77.39%
10 parties, 10,000 records	6234.8	3512.4	1847.5	1398.2	77.57%
Arithmetic mean	2715.6	1530.8	804.4	611.2	77.49%

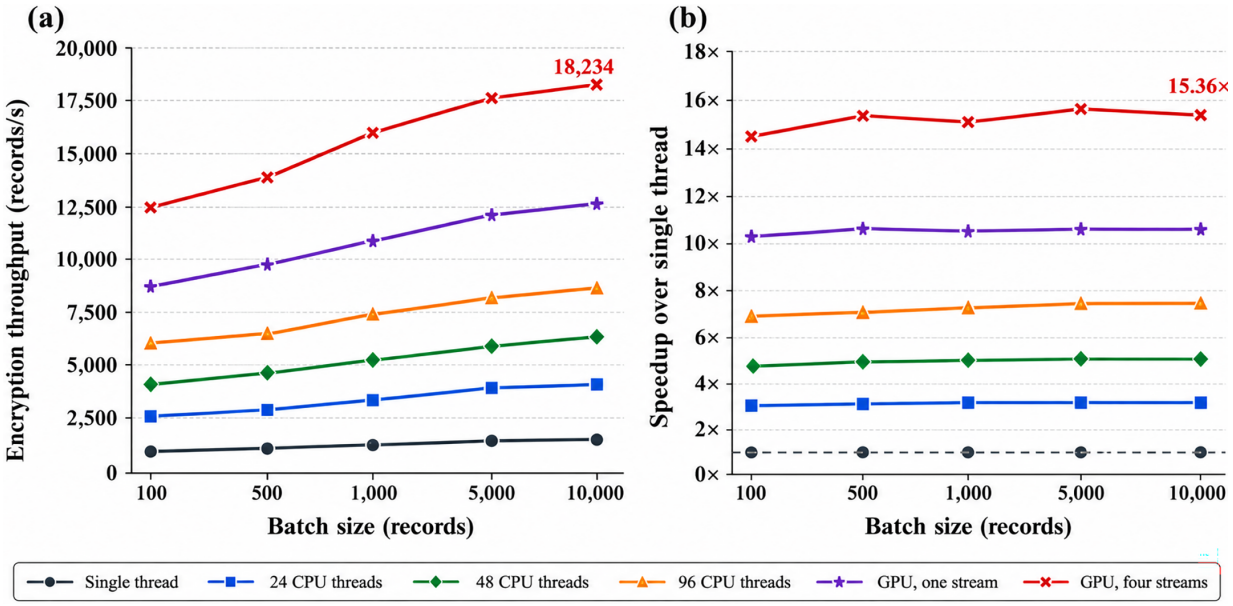


FIGURE 3 | Encryption performance across batch sizes and execution modes: (a) encryption throughput and (b) acceleration relative to single-thread execution. The four-stream GPU configuration reached 18,234 records/s and a maximum speedup of 15.36× at a batch size of 10,000 records.

shares. To ensure the results of the revised experiment could be traced back to the original experiment, the three workload conditions were kept.

The key-generation latency results for the hybrid CPU–GPU implementation are consistent and shown in Table 8 for the different scales evaluated.

The mean latency was reduced from 2715.6 to 611.2 ms, which is a 77.49% reduction. The improvements were not sensitive to one of the workloads, ranging from 76.47% to 77.57% across the three configurations. Polynomial sampling and lattice arithmetic were performed on the GPUs, and protocol orchestration, commitment verification, and inter-node coordination were performed on the CPU.

The performance for encryption only was then tested as the batch size was increased. These measurements do not include secure function evaluation, verification, reconstruction, and coercion-state processing.

The throughput of the encryption should be evident in Figure 3, which shows it was close to saturation for larger batches. GPU occupancy was 76.3%, while the measured device memory bandwidth was 92.4%. When the bandwidth was not fully utilized, the memory movement was the main constraint, as shown by the higher bandwidth utilization. In the case of packed-message homomorphic execution and functional bootstrapping, the benefit of parallel arithmetic is influenced by the data layout and memory traffic [9, 10, 33].

The value of 18,234 records/s is only for encryption throughput. It is not the same as complete-protocol throughput as it does not cover authorization, secure evaluation, active checks, output reconstruction, or state protection.

4.3 | Reconciled End-to-End Latency and Throughput

The throughput inconsistency that the reviewer found was due to assuming record throughput and complete protocol invocations were the same. The new analysis will be based on processing 1000 records per batch, which is considered one invocation.

The optimized-core invocation took 162.5 ms when invoked by a two-party. This equals 6.154 full-batch calls per second. The record throughput for each invocation was 6154 records/s since there were 1000 records per invocation. The previously reported number of 6154 was not the actual number of protocol executions per second, but rather the number of processed records per second.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Phase-wise latency of the complete proposed protocol.

Parties	Key generation (ms)	Encryption (ms)	Secure evaluation (ms)	Active verification (ms)	Reconstruction (ms)	Coercion-state handling (ms)	Full latency (ms)
2	76.4	43.8	21.4	8.2	20.9	5.1	175.8
3	95.3	58.7	32.1	10.4	32.5	7.3	236.3
5	134.6	92.4	61.8	16.2	60.1	12.4	377.5
8	203.5	142.8	104.3	25.1	93.1	19.5	588.3
10	251.4	178.3	134.7	31.2	98.4	23.1	717.1

TABLE 10 | Reconciled the latency and throughput of the complete protocol.

Parties	Sequential full protocol (ms)	Optimized FE–SFE core (ms)	Complete proposed protocol (ms)	Reduction from sequential	Complete invocations/s	Records/s
2	302.1	162.5	175.8	41.81%	5.688	5688
3	400.9	218.6	236.3	41.06%	4.232	4232
5	648.7	348.9	377.5	41.81%	2.649	2649
8	998.8	543.7	588.3	41.10%	1.700	1700
10	1228.5	662.8	717.1	41.63%	1.395	1395

Active verification and coercion-state handling are not shown separately in the revised phase decomposition but were shown separately in the original timing table.

The phase values in Table 9 are additive. For instance, the five-party configuration took 134.6 ms to generate the key, 92.4 ms to encrypt, 61.8 ms to securely evaluate, 16.2 ms to actively verify, 60.1 ms to reconstruct, and 12.4 ms to process the coercion state, resulting in a total latency of 377.5 ms.

The key generation phase remained the largest individual phase. As the number of parties grew, their relative contribution diminished, and secure evaluation and active verification grew in relative importance as more authenticated values and intermediate results were to be verified.

Even with the inclusion of active verification and coercion-state handling, the complete framework still showed a 41.06%–41.81% reduction compared to sequential execution, as indicated in Table 10. The optimized-core throughput was 6154 records/s for the two-party case, and 5688 records/s for the complete security-enabled framework. The optimized-core latency of 662.8 ms was used for 10 parties, and the latency of the full protocol was 717.1 ms, with 1509 records/s and 1395 records/s, respectively.

The earlier contradiction is eliminated with these reconciled results. The record-throughput numbers of 6154 and 1509 are not thousands of protocol invocations per second, but rather 1000-record invocations.

The throughput units should be shown in Figure 4. This visual separation helps avoid confusion between the number of records in a batch and the number of independent protocol calls.

4.4 | Malicious-Behavior Detection and Security Overhead

The evaluation for active security was based on eight intentional protocol violations. About 10,000 sessions were independently generated, and each attack class was injected into the sessions. A test was deemed to be safely handled if the deviation was detected and the test was aborted before an invalid result was released.

Table 11 reports the detection outcomes and the time required to identify each attack class.

No injected deviation resulted in an accepted invalid output. The evaluated attack distribution had a one-sided 99% upper confidence bound on the probability of an attack being undetected of approximately 0.00576% based on 80,000 trials with no attacks detected.

Replay and duplicate-session attempts were rejected the fastest due to validation failures before costly cryptographic operations were performed. The altered ciphertext coefficients needed to be committed to and checked against the coefficients and were slightly longer. The highest detection latency was obtained when the party terminated prematurely, as this was detected when a synchronization timeout was reached, which was bounded.

With a five-party reference configuration, the optimized-core latency rose from 348.9 to 365.1 ms with the activation of the verification. This overhead was 4.64%. The total time for the coercion-state handling was 377.5 ms, which was still 41.81% faster than the sequential approach. These results show that the primary acceleration did not come from the active-security layer; the other three factors (batching, GPU processing, and communication overlap) remained the most significant sources of acceleration.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

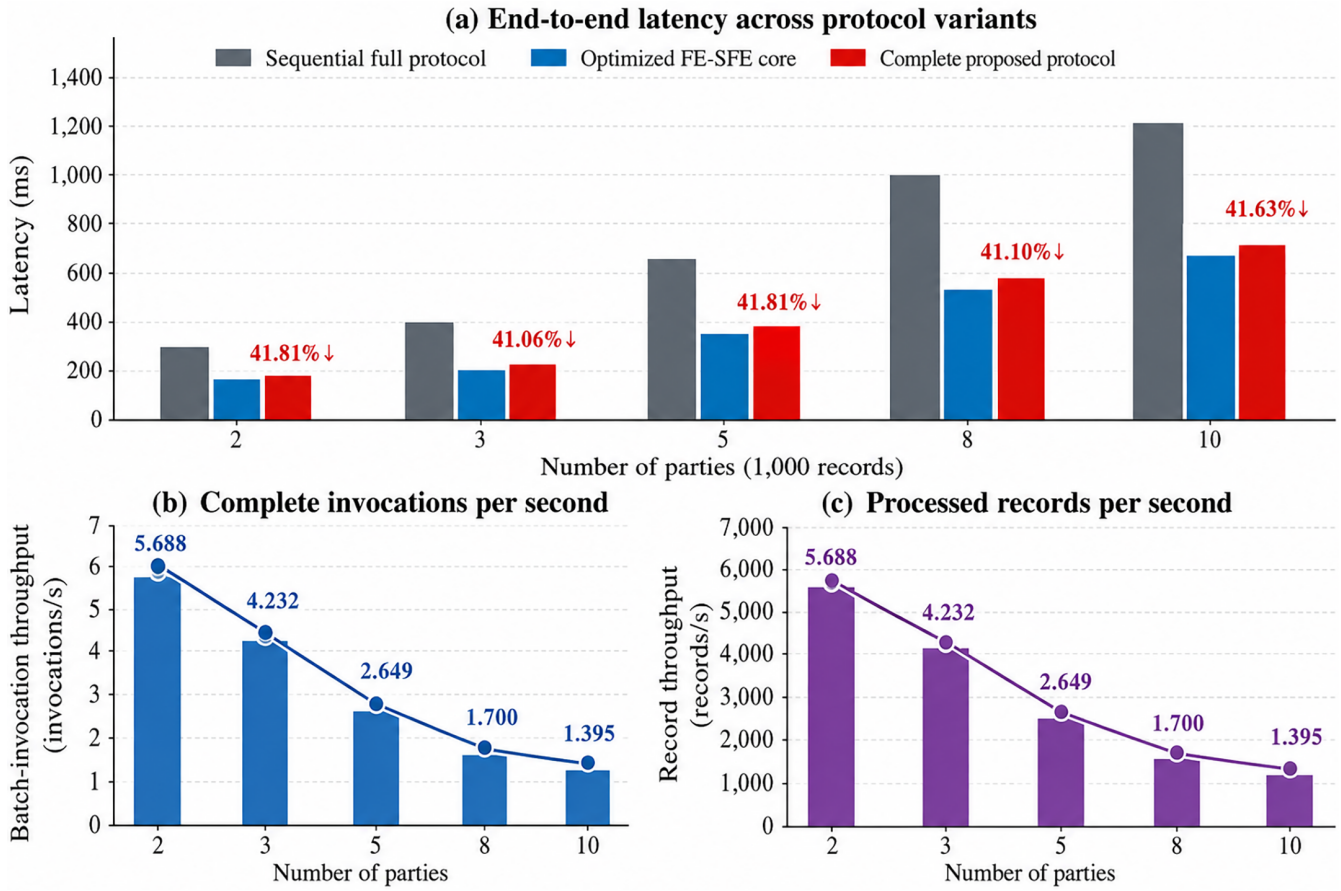


FIGURE 4 | Reconciled relationship between complete-protocol latency, batch-invocation throughput, and record throughput. (a) End-to-end latency across protocol variants. (b) Complete invocations per second. (c) Processed records per second.

TABLE 11 | Detection of malicious protocol deviations.

Injected behavior	Trials	Detected or safely aborted	Invalid outputs accepted	Detection rate	Median detection latency
Forged authentication tag	10,000	10,000	0	100.000%	0.31 ms
Inconsistent Beaver triple	10,000	10,000	0	100.000%	0.46 ms
Modified ciphertext coefficient	10,000	10,000	0	100.000%	0.73 ms
Replayed protocol message	10,000	10,000	0	100.000%	0.12 ms
Duplicate session identifier	10,000	10,000	0	100.000%	0.10 ms
Unauthorized function digest	10,000	10,000	0	100.000%	0.18 ms
Incorrect reconstruction share	10,000	10,000	0	100.000%	0.34 ms
Premature party termination	10,000	10,000	0	100.000%	20.6 ms
Total	80,000	80,000	0	100.000%	—

The small verification overhead is like the recent studies on constant-round MPC and publicly verifiable secure computation, which batch the operations of authentication and proof to minimize the online interaction [12, 41, 42, 46, 47].

4.5 | Coercion-Resistance Validation

Coercion resistance was evaluated under the bounded post-execution threat model defined in Section 3.7. Each genuine retained state was paired with an alternative state generated from the same encrypted masked value but a different claimed plaintext. Training and test transcripts were separated by protocol session to prevent memorization of session-specific artifacts.

The evaluation used 100,000 retained states, comprising 50,000 genuine and 50,000 alternative states. Logistic regression, random forest, and a shallow multilayer perceptron were used as transcript distinguishers. Input features included message count, padded message length, stage timing, retained-state size, and non-secret protocol metadata.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

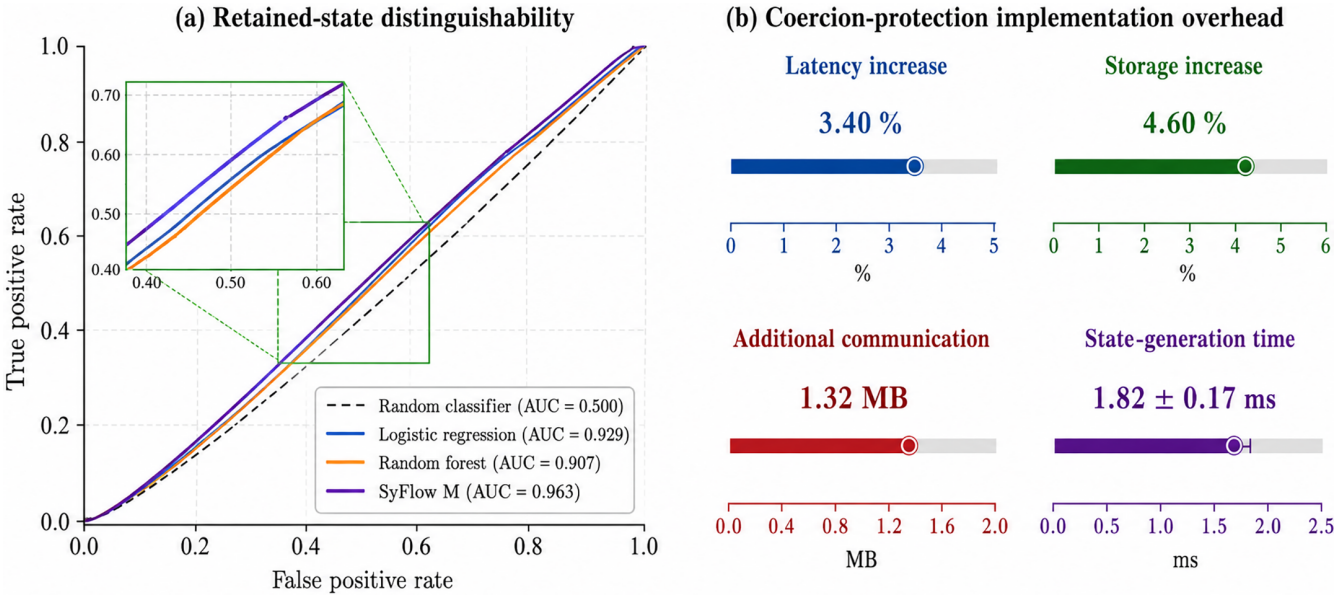


FIGURE 5 | Coercion-resistance evaluation and implementation overhead: (a) presents ROC curves for the three retained-state distinguishers and (b) reports the additional latency, storage, communication, and state-generation time introduced by coercion protection.

Generating an alternative retained state required an average of 1.82 ms with a standard deviation of 0.17 ms. In the five-party configuration, coercion-state handling increased latency from 365.1 to 377.5 ms, corresponding to a 3.40% increase. Retained-state storage increased by 4.6%, while mask refresh and session-state evidence added 1.32 MB of communication in the 10-party configuration.

The result shown in Figure 5 is limited to the stated threat model. It provides evidence for post-execution transcript equivocation after mask refresh and erasure. It does not cover pre-erasure memory acquisition, physical side-channel monitoring, or compromise of the reconstruction threshold. Function-hiding and private-function-evaluation research similarly distinguish transcript privacy from stronger physical or adaptively compromised settings [7, 21, 25, 36].

4.6 | Communication and Resource Analysis

Communication was measured directly from transmitted protocol bytes. Share distribution remained the largest component because masked inputs and authenticated values had to be delivered to multiple evaluators. Compression and batching reduced the core communication volume from 243.33 to 84.28 MB, a 65.37% reduction. After adding 6.49 MB for authentication, verification, replay protection, and coercion-state exchange, the complete protocol transferred 90.77 MB. The final communication requirement, therefore, remained 62.70% below the unoptimized transfer volume.

The security-related messages did not eliminate the benefit of communication optimization. This distinction matters in multi-party systems because methods that improve cryptographic computation while increasing communication can perform poorly once bandwidth or latency is constrained. Similar communication–computation trade-offs have been reported in multi-party federated learning, secret sharing, and packed homomorphic execution [1, 7, 13, 21].

As can be seen in Table 12, the maximum CPU utilization was not associated with the minimum latency. The multithreaded CPU configuration consumed 88.4% of the CPU resources and was 20.3% slower than the full hybrid configuration. GPU offloading freed CPU resources for authentication, communication, and output reconstruction.

The number of memory buffers used for the ciphertext, communication and verification buffers meant that peak memory usage was 1.0 GB higher than with sequential execution. The improvement was not as significant as the latency improvement achievable with the overlapped pipeline.

4.7 | Component-Wise Ablation Analysis

A controlled ablation experiment was used to evaluate the contribution of each optimization and security component and is displayed in Table 13. The five-party workload, 5000 records, 1000-record batches, 128-bit security, and physical cluster were all the same for all configurations.

The greatest single optimization benefit was obtained from the batching. It was found that removing it increased latency by 19.6%. The latency for the 10-Gbps network was 14.1% higher when asynchronous communication was disabled and 7.4% higher when compression was disabled.

Removing active-security checks decreased latency by 4.5%, and removing coercion protection decreased latency by 3.3%. These differences validate that the main performance improvement was not due to a reduction in the security model. The full system was

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 12 | Resource profile for the five-party, 5000-record workload.

Execution mode	Latency (ms)	CPU utilization	GPU occupancy	Peak memory (GB)	Network transfer per invocation (MB)
Sequential CPU	648.7	12.6%	—	10.8	68.99
Multithreaded CPU	454.2	88.4%	—	11.2	25.74
GPU, one stream	418.6	59.2%	58.7%	11.6	25.74
Complete the proposed framework	377.5	66.8%	76.3%	11.8	25.74

TABLE 13 | Reports the end-to-end effect of disabling each component individually.

Configuration	Latency (ms)	Records/s	Speedup over a sequential CPU	Latency change relative to the complete framework
Sequential CPU	648.7	1542	1.00×	+71.8%
Multithreaded CPU	454.2	2202	1.43×	+20.3%
GPU, one stream	418.6	2389	1.55×	+10.9%
Without batching	451.6	2214	1.44×	+19.6%
Without compression	405.3	2467	1.60×	+7.4%
Without asynchronous communication	430.8	2321	1.51×	+14.1%
Without active security checks	360.7	2772	1.80×	−4.5%
Without a coercion-resistance layer	365.1	2739	1.78×	−3.3%
Complete the proposed framework	377.5	2649	1.72×	Reference

1.72 times faster than running sequentially on a CPU, and offered the full security feature set.

4.8 | Statistical Significance and Effect Size

Statistical analysis used individual measurements rather than the aggregate values presented in the tables. Latency distributions exhibited positive skew in high-party and delayed-network conditions; log-transformed latency was therefore used for the factorial analysis.

For the five-party reference configuration, complete-protocol latency was 377.5 ± 9.8 ms. The corresponding 99% confidence interval was 373.8–381.2 ms. Record throughput was 2649 ± 68 records/s, with a 99% confidence interval of 2623–2675 records/s. Ten-party communication was 90.77 ± 1.84 MB.

Figure 6 shows the magnitude and direction of those effects. Panel (a) presents the adjusted latency difference between each controlled configuration and the complete proposed framework. Positive values indicate that the comparison configuration required more processing time; negative values occur only when a security mechanism is applied.

4.9 | Controlled Comparison With Existing Configurations

Quantitative comparison was restricted to configurations evaluated under the same hardware, workload, party count, security target, batch size, and network condition. Values quoted from studies executed on unrelated hardware were not used to calculate speedup.

As shown in Table 14, the controlled comparison used five parties, 5000 records, 1000-record batches, and 128-bit security.

Within security-aligned configurations, the proposed framework reduced latency by 16.89% relative to the CPU-only hybrid and by 8.44% relative to authenticated SFE. Compared with the packed-HE reference, it reduced latency by 11.92% and peak memory by 15.11%.

The decentralized IPFE reference remained 13.77% faster than the complete framework. This outcome is expected because it performs only linear evaluation under a semi-honest model. It does not execute authenticated Boolean operations, verify malicious behavior, or process coercion states. Presenting it as a universally inferior baseline would therefore be misleading.

The novelty of the proposed framework lies in achieving competitive latency while combining a broader security and functional scope. Decentralized and function-hiding FE methods provide efficient restricted computation, but they do not necessarily include malicious-secure circuit execution or post-execution coercion resistance [5, 7, 21, 28, 29]. Similarly, packed-HE and secure federated learning systems must be compared based on the adversarial models they support and their verification capabilities, rather than on throughput alone [1, 2, 32].

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

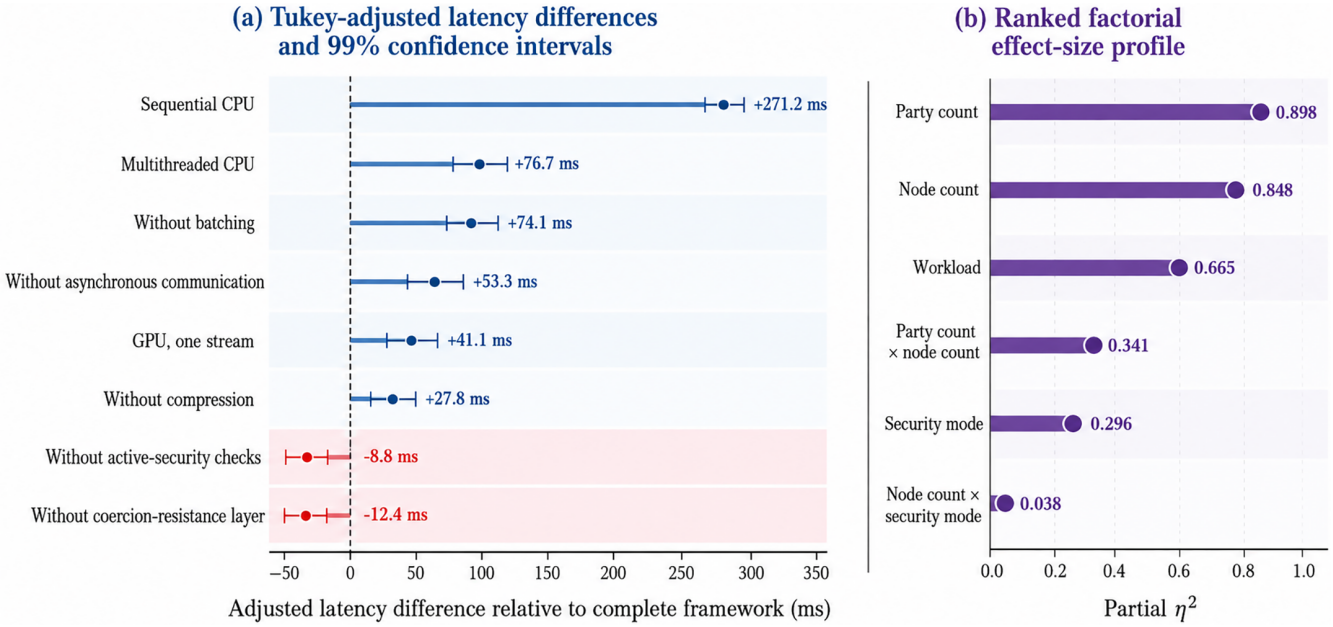


FIGURE 6 | Statistical significance and effect-size profile of the proposed FE–SFE framework. Panel (a) presents Tukey-adjusted mean latency differences relative to the complete proposed framework, with 99% confidence intervals. Positive values favor the proposed framework because the corresponding comparison required greater latency. Negative values identify faster configurations obtained by removing active-security or coercion-resistance mechanisms. Panel (b) reports partial eta-squared values for the principal experimental factors and interactions. Party count and node count produced the largest effects on end-to-end latency, whereas the node-count-by-security-mode interaction produced the smallest effect.

TABLE 14 | Controlled comparison with functionally aligned reference configurations.

Configuration	Security and functional scope	Latency (ms)	Complete invocations/s	Records/s	Peak memory (GB)	Communication (MB)
Decentralized IPFE reference	Semi-honest; linear evaluation only	331.8	3.014	3014	9.7	71.50
Packed-HE reference	Semi-honest; arithmetic evaluation	428.6	2.333	2333	13.9	128.40
Authenticated-SFE reference	Malicious with abort; no function hiding	412.3	2.425	2425	10.9	116.70
CPU-only hybrid FE–SFE	Malicious with abort and coercion layer	454.2	2.202	2202	11.2	94.80
Complete the proposed framework	Function hiding, malicious security with abort, bounded coercion resistance	377.5	2.649	2649	11.8	90.77

4.10 | Experimental Scalability and Claim Boundaries

Scalability was evaluated only over the measured range of 2–10 physical nodes. Efficiency was calculated relative to the two-node execution time for the same workload. The 75% threshold defined in Section 3.12 was used to identify the efficient scaling region.

As shown in Figure 7, 10 nodes were not slower than eight nodes; rather, the marginal return from the additional resources was lower. Profiling attributed the change to synchronization, distributed verification, and final aggregation. Communication and synchronization represented approximately one-third of execution time at 10 nodes.

5 | Discussion

The results show that the proposed FE–SFE framework provides a realistic compromise between efficiency and enhanced security guarantees. The end-to-end latency decreased by over 41% compared to sequential execution, with GPU acceleration, batching, compression, and asynchronous communication incurring moderate overhead, while active verification and coercion protection incurred negligible overhead. This suggests that the observed improvement was mainly due to optimization of the architecture and not to a reduction of the opposition model.

The framework also ensured functional correctness in all the workloads tested and prevented unauthorized, incomplete, or

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

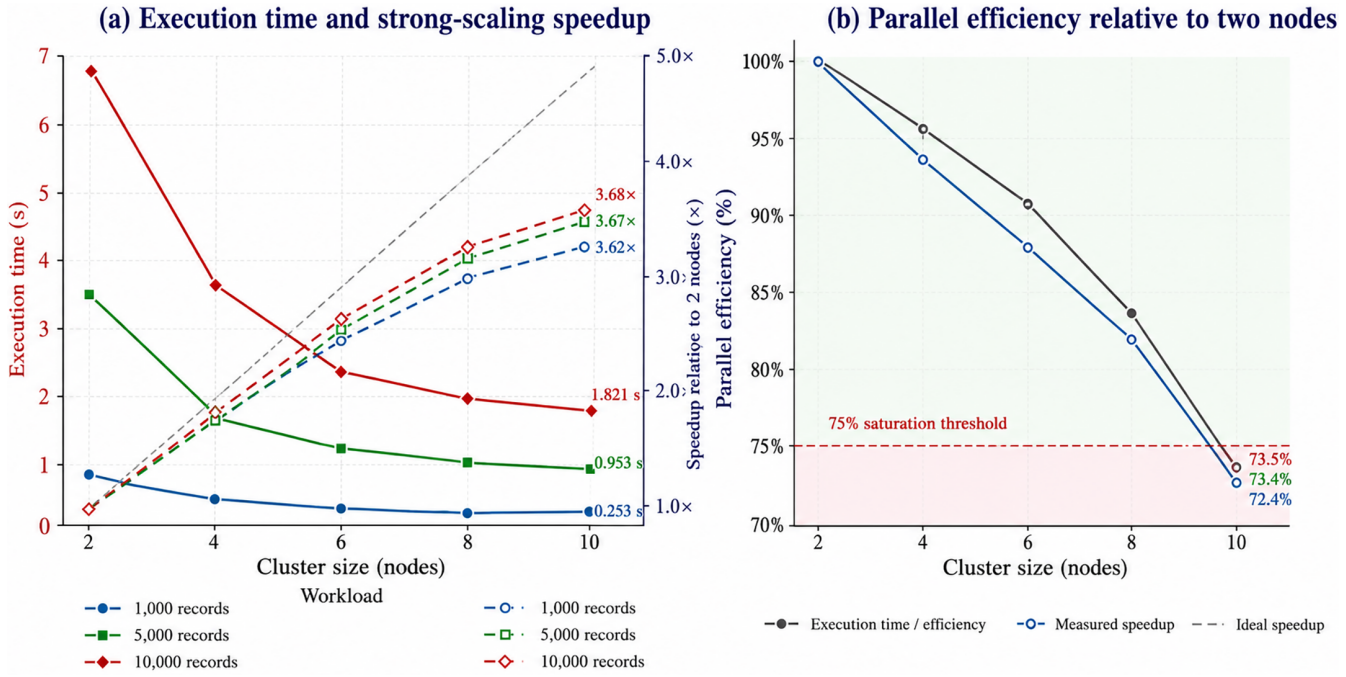


FIGURE 7 | Experimental speedup and parallel efficiency across cluster sizes. Panel (a) presents execution time and speedup for workloads containing 1000–10,000 records. Panel (b) presents efficiency relative to two nodes. The horizontal 75% line marks the predefined saturation threshold.

malicious executions. The advantage of its performance was most pronounced when compared to security-aligned configurations, while the semi-honest IPFE baseline was still faster due to its limited functional and threat scope. This separation highlights the importance of benchmarking privacy-preserving systems with the same security assumptions [5, 42].

The communication and synchronization overheads decreased the parallel efficiency below the desired 75% for more than 8 nodes, but the efficiency was still good up to 8 nodes. The results thus indicate that deployment on medium scale distributed infrastructures is possible, but for larger clusters, communication-aware scheduling and more efficient aggregation are needed. The coercion-resistance results are only applicable to the post-execution transcript model and do not apply to physical side channels, pre-erasure memory compromise, or threshold-level collusion.

6 | Conclusion and Future Scope

In this study, a hybrid FE-SFE framework was introduced that combines function-restricted data disclosure, authenticated secure computation, malicious-security verification, and bounded post-execution coercion resistance. The optimized implementation achieved an end-to-end latency of about 41% less than the sequential implementation and maintained the functional correctness of the implementation. The primary performance improvements were due to GPU acceleration, batch processing, compression and asynchronous communication, while active verification and coercion protection added only a small amount of overhead. The framework was also able to identify all the malicious protocol violations evaluated and achieved efficient strong scaling up to 8 physical nodes. The results show that it is possible to achieve more stringent privacy and verification guarantees without making multi-party computation impractical.

The framework should be tested in larger and geographically spread clusters in future to assess the framework in more variable networks with synchronization delays. There are other issues that need to be explored, such as adaptive corruption, threshold-level collusion, pre-erasure memory compromise, and physical side-channel leakage. Proactive share refresh, dynamic function-policy updates, heterogeneous accelerator support, and communication-aware scheduling are among the other enhancements that could be added. The framework's practical applicability would be further strengthened by formalizing its guarantees and validating it in deployments such as in healthcare, finance, and federated learning [23, 42].

Funding

The authors have nothing to report.

Ethics Statement

The authors have nothing to report.

Consent

All authors have read and agreed to the published version of the manuscript.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The deterministic cryptographic benchmark vectors, known-answer outputs, party-partition manifests, attack-injection specifications, and

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

analysis configuration used for the revised evaluation are provided in the accompanying supplementary archive. The Adult dataset is publicly available from the UCI Machine Learning Repository under DOI 10.24432/C5XW20, and MNIST is publicly available from the database maintained by Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. Retrieval and deterministic preprocessing scripts, source identifiers, random seeds, and integrity checks are included in the archive.

References

1. M. Fan, Z. Zhang, Z. Li, et al., "SecureVFL: Privacy-Preserving Multi-Party Vertical Federated Learning Based on Blockchain and RSS," *Digital Communications and Networks* 11, no. 3 (2025): 837–849, https://doi.org/10.1016/j.dcan.2024.07.008.

2. Y. Huang, W. Wang, X. Zhao, et al., "EFMVFL: An Efficient and Flexible Multi-Party Vertical Federated Learning Without a Third Party," *ACM Transactions on Knowledge Discovery From Data* 18, no. 3 (2024): 51, https://doi.org/10.1145/3627993.

3. R. Naz, M. Almaiah, and A. Alqutaish, "A Hybrid Blockchain-Based Framework for Adaptive Cyber-Risk Prediction and Multi-Layer Threat Mitigation in Enterprise Networks," *Journal of Cybersecurity and Privacy* 6, no. 3 (2026): 85, https://doi.org/10.3390/jcp6030085.

4. P. Vidyullatha, A. Alqutaesh, G. Alradwan, and M. Obedat, "A Hybrid Fuzzy–Deep Learning Framework for Real-Time Cyber-Attack Detection in Smart Energy Grids," *International Journal of Data and Network Science* 10 (2026): 1541–1552, https://doi.org/10.5267/j.ijdns.2026.2.007.

5. Y. Chang, K. Zhang, J. Gong, and H. Qian, "Privacy-Preserving Federated Learning via Functional Encryption, Revisited," *IEEE Transactions on Information Forensics and Security* 18 (2023): 1855–1869, https://doi.org/10.1109/TIFS.2023.3255171.

6. V. Cini, S. Ramacher, D. Slamanig, C. Striecks, and E. Tairi, "(Inner-Product) Functional Encryption With Updatable Ciphertexts," *Journal of Cryptology* 37 (2024): 8, https://doi.org/10.1007/s00145-023-09486-y.

7. Y. Li, J. Wei, F. Guo, Y. Xiang, and X. Chen, "Function-Hiding Multi-Client Inner-Product Functional Encryption Without Pairings for Large Space," *IEEE Transactions on Dependable and Secure Computing* 22, no. 5 (2025): 5786–5797, https://doi.org/10.1109/TDSC.2025.3576164.

8. H. Yang and C. Peng, "CP-IPFE: Ciphertext-Policy Based Inner Product Functional Encryption," *IEEE Transactions on Information Forensics and Security* 19 (2024): 5419–5433, https://doi.org/10.1109/TIFS.2024.3396395.

9. L. Bergerat, C. Bonte, B. R. Curtis, J.-B. Orfila, P. Paillier, and S. Tap, "Sharing the Mask: TFHE Bootstrapping on Packed Messages," *IACR Transactions on Cryptographic Hardware and Embedded Systems* 4 (2025): 925–971, https://doi.org/10.46586/tches.v2025.i4.925-971.

10. J. Kim, "Efficient Homomorphic Integer Computer From CKKS," *IACR Transactions on Cryptographic Hardware and Embedded Systems* 4 (2025): 873–898, https://doi.org/10.46586/tches.v2025.i4.873-898.

11. T. Nakai and K. Shinagawa, "Secure Multi-Party Computation With Legally-Enforceable Fairness," *International Journal of Information Security* 23 (2024): 3609–3623, https://doi.org/10.1007/s10207-024-00898-w.

12. Y. Wang, K. Xiong, Y. Tang, L. Yang, J. Zhang, and X. Yan, "More Efficient Constant-Round Secure Multi-Party Computation Based on Optimized Tiny-OT and Half-Gate," *Journal of Information Security and Applications* 79 (2023): 103650, https://doi.org/10.1016/j.jisa.2023.103650.

13. Y. Yang, B. Li, Q. Liu, et al., "PlainDrop: Practical Asynchronous Proactive Secret Sharing With Silent Setup," *IEEE Internet of Things Journal* 13, no. 5 (2026): 8853–8867, https://doi.org/10.1109/JIOT.2025.3641700.

14. O. Ahmed, C. Gouert, and N. G. Tsoutsos, "PEEV: Parse Encrypt Execute Verify—A Verifiable FHE Framework," *IEEE Access* 12 (2024): 94673–94689, https://doi.org/10.1109/ACCESS.2024.3424420.

15. J. Liu, X. Li, X. Liu, et al., "Privacy-Preserving and Verifiable Outsourcing Linear Inference Computing Framework," *IEEE Transactions on Services Computing* 16, no. 6 (2023): 4591–4604, https://doi.org/10.1109/TSC.2023.3332933.

16. X. Liu, G. Yang, W. Susilo, K. He, R. H. Deng, and J. Weng, "Privacy-Preserving Multi-User Outsourced Computation for Boolean Circuits," *IEEE Transactions on Information Forensics and Security* 18 (2023): 4929–4943, https://doi.org/10.1109/TIFS.2023.3301734.

17. G. Ling, F. Tang, C. Cai, et al., "P^2FRPSI: Privacy-Preserving Feature Retrieved Private Set Intersection," *IEEE Transactions on Information Forensics and Security* 19 (2024): 2201–2216, https://doi.org/10.1109/TIFS.2023.3343973.

18. R. Wang, J. Zhou, Z. Cao, X. Dong, and K.-K. R. Choo, "Updatable Private Set Intersection With Forward Privacy," *IEEE Transactions on Information Forensics and Security* 19 (2024): 8573–8586, https://doi.org/10.1109/TIFS.2024.3461475.

19. W. Wang, Z. Jia, M. Xu, and S. Li, "FPMRQ: Fully Privacy-Preserving Multidimensional Range Queries on Encrypted Data," *IEEE Internet of Things Journal* 11, no. 7 (2024): 12362–12376, https://doi.org/10.1109/JIOT.2023.3334615.

20. G. Xu, W. Ji, Y. Wang, X. Shi, Q. Huang, and Y. Gan, "Searchable Encryption Algorithm Based on Key Aggregation of Multiple Data Owners in Data Sharing," *Journal of Information Security and Applications* 78 (2023): 103600, https://doi.org/10.1016/j.jisa.2023.103600.

21. Y. Li, N. Yan, J. Chen, et al., "FedPHE: A Secure and Efficient Federated Learning via Packed Homomorphic Encryption," *IEEE Transactions on Dependable and Secure Computing* 22, no. 5 (2025): 5448–5463, https://doi.org/10.1109/TDSC.2025.3567301.

22. Y. Pan, Z. Chao, W. He, Y. Jing, L. Hongjia, and W. Liming, "FedSHE: Privacy Preserving and Efficient Federated Learning With Adaptive Segmented CKKS Homomorphic Encryption," *Cybersecurity* 7 (2024): 40, https://doi.org/10.1186/s42400-024-00232-w.

23. S. Baranski and J. Szymanski, "Federated Distributed Key Generation," 177 (2026): 108226, https://doi.org/10.1016/j.future.2025.108226.

24. I. Levi and C. Hazay, "Garbled Circuits From an SCA Perspective: Free XOR Can Be Quite Expensive . . . ," *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2023, no. 2 (2023): 54–79, https://doi.org/10.46586/tches.v2023.i2.54-79.

25. V. Ahuja, I. Kishor, A. Alqutaish, R. Shehab, and M. Obeidat, "Mitigating Information Leakage Risks in Secure Multiparty Computation Through Function Hiding," *Journal of Cyber Security and Risk Auditing* 1 (2026): 38–72, https://doi.org/10.63180/jcsra.thestap.2026.1.3.

26. M. Almaiah, A. Alqutaish, R. Shehab, and M. Obeidat, "Behavior-Aware Cybersecurity Using Artificial Intelligence and Cryptographic Intelligence," *International Journal of Data and Network Science* 10, no. 2 (2026): 699–722.

27. M. Almaiah, U. Mamodiya, I. Kishor, A. Alqutaish, and M. C. Ghanem, "AI-Driven Optimization Frameworks for Scalable and Energy-Efficient Blockchain Networks," *Telematics and Informatics Reports* 23 (2026): 100347, https://doi.org/10.1016/j.teler.2026.100347.

28. X. Qian, H. Li, M. Hao, G. Xu, H. Wang, and Y. Fang, "Decentralized Multi-Client Functional Encryption for Inner Product With Applications to Federated Learning," *IEEE Transactions on Dependable and Secure Computing* 21, no. 6 (2024): 5781–5796, https://doi.org/10.1109/TDSC.2024.3386357.

29. J. Han, L. Chen, A. Hu, L. Chen, and J. Li, "Privacy-Preserving Decentralized Functional Encryption for Inner Product," *IEEE Transactions on Dependable and Secure Computing* 21, no. 4 (2024): 1680–1694, https://doi.org/10.1109/TDSC.2023.3288109.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

30. J. Han, L. Chen, W. Susilo, L. Chen, and G. Wu, "DFE-IP: Delegatable Functional Encryption for Inner Product," *Information Sciences* 647 (2023): 119425, https://doi.org/10.1016/j.ins.2023.119425.

31. X. Che, L. Liu, B. Wang, et al., "Multi-Key Homomorphic Encryption With Tightened RGSW Ciphertexts Without Relinearization for Ciphertexts Product," *Journal of King Saud University, Computer and Information Sciences* 35, no. 10 (2023): 101794, https://doi.org/10.1016/j.jksuci.2023.101794.

32. Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "SecFed: A Secure and Efficient Federated Learning Based on Multi-Key Homomorphic Encryption," *IEEE Transactions on Dependable and Secure Computing* 21, no. 4 (2024): 3817–3833, https://doi.org/10.1109/TDSC.2023.3336977.

33. H. Xia and M. Wang, "Efficient and Compact Full-Domain Functional Bootstrapping via Subring Folding," *IACR Transactions on Cryptographic Hardware and Embedded Systems* 4 (2025): 737–762, https://doi.org/10.46586/tches.v2025.i4.737-762.

34. J. Domingo-Ferrer and J. Manjón, "Circuit-Free General-Purpose Multi-Party Computation via Co-Utile Unlinkable Outsourcing," *IEEE Transactions on Dependable and Secure Computing* 20, no. 1 (2023): 539–550, https://doi.org/10.1109/TDSC.2021.3138826.

35. F. van Daalen, L. Ippel, A. Dekker, and I. Bermejo, "Privacy Preserving n-Party Scalar Product Protocol," *IEEE Transactions on Parallel and Distributed Systems* 34, no. 4 (2023): 1060–1066, https://doi.org/10.1109/TPDS.2023.3238768.

36. K. Ji, B. Zhang, T. Lu, and K. Ren, "Multi-Party Private Function Evaluation for RAM," *IEEE Transactions on Information Forensics and Security* 18 (2023): 1252–1267, https://doi.org/10.1109/TIFS.2023.3236457.

37. X. Yang, J. Chen, Y. Li, et al., "Fregata: Fast Private Inference With Unified Secure Two-Party Protocols," *IEEE Transactions on Information Forensics and Security* 19 (2024): 8472–8484, https://doi.org/10.1109/TIFS.2024.3444327.

38. Z. Cao, "Quantum Private Function Evaluation," *New Journal of Physics* 25 (2023): 103027, https://doi.org/10.1088/1367-2630/acffec.

39. T. Mohanty and S. K. Debnath, "An Information-Theoretically Secure Quantum Multiparty Private Set Intersection," *Journal of Information Security and Applications* 78 (2023): 103623, https://doi.org/10.1016/j.jisa.2023.103623.

40. X. Chen and L. F. Zhang, "Publicly Verifiable Homomorphic Secret Sharing for Polynomial Evaluation," *IEEE Transactions on Information Forensics and Security* 18 (2023): 4609–4624, https://doi.org/10.1109/TIFS.2023.3298258.

41. X. Li, H. Wang, Z. Li, et al., "Publicly Verifiable Secure Multi-Party Computation Framework Based on Bulletin Board," *IEEE Transactions on Services Computing* 17, no. 4 (2024): 1698–1711, https://doi.org/10.1109/TSC.2024.3380258.

42. K. Gai, D. Wang, J. Yu, L. Zhu, and W. Meng, "A Scheme of Robust Privacy-Preserving Multi-Party Computation via Public Verification," *IEEE Transactions on Dependable and Secure Computing* 22, no. 5 (2025): 4896–4910, https://doi.org/10.1109/TDSC.2025.3555284.

43. Y. Zhang, H. Geng, L. Su, S. He, and L. Lu, "An Efficient Polynomial-Based Verifiable Computation Scheme on Multi-Source Outsourced Data," *Scientific Reports* 14 (2024): 8512, https://doi.org/10.1038/s41598-024-53267-x.

44. L. Harn, C. Hsu, Z. Xia, H. Xu, S. Zeng, and F. Pang, "Simple and Efficient Threshold Changeable Secret Sharing," *Journal of Information Security and Applications* 77 (2023): 103576, https://doi.org/10.1016/j.jisa.2023.103576.

45. Y. Yang, Y. Yang, X. Chen, X. Dong, Z. Cao, and J. Shen, "DMPSI: Efficient Scalable Delegated Multiparty PSI and PSI-CA With Oblivious PRF," *IEEE Transactions on Services Computing* 17, no. 2 (2024): 497–508, https://doi.org/10.1109/TSC.2024.3356667.

46. X. Wang, J. Wu, and J. Wang, "One for All: Efficient Verifiable Dynamic Multi-User Searchable Encryption in the Presence of Corrupted Users," *Journal of King Saud University, Computer and Information Sciences* 35, no. 8 (2023): 101703, https://doi.org/10.1016/j.jksuci.2023.101703.

47. Y. Wang, Y. Zeng, S. Li, Y. Zhang, Z. Zhou, and Y. Tong, "Efficient and Private Federated Trajectory Matching," *IEEE Transactions on Knowledge and Data Engineering* 36, no. 12 (2024): 8079–8092, https://doi.org/10.1109/TKDE.2024.3424411.

48. L. Zhang, Z. Ou, C. Hu, H. Kan, and J. Zhang, "Data Sharing in the Metaverse With Key Abuse Resistance Based on Decentralized CP-ABE," *IEEE Transactions on Computers* 74, no. 3 (2025): 901–914, https://doi.org/10.1109/TC.2024.3512177.

49. Y. Xue, L. Liu, Y. Luo, B. Sun, and S. Fu, "FLUTE: FSS-Based Secure Two-Party LLM Inference Using Partial Transformer Encryption," *IEEE Transactions on Dependable and Secure Computing* 23, no. 1 (2026): 685–702, https://doi.org/10.1109/TDSC.2025.3610646.

50. U. Mamodiya, D. Sinha, I. Kishor, A. A. Syed, and A. Routray, "An AI–Blockchain-Based Network Optimization Framework for Energy-Efficient Computing Systems," *IET Networks* 15 (2026): e70029, https://doi.org/10.1049/ntw2.70029.

15320634, 2026, 18, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cpe.70949 by National Institute Of Technology, Durgapur, Wiley Online Library on [04/09/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License